ANANTA OCR: A Unified Multimodal Framework for Multilingual Document Intelligence with Layout-Aware Recognition and Differentiable Sparse Attention

Front Matter

Author Information

- **ASHUTOSH YADAV**
  Department of Computer Science and Engineering
  Email: ASHUTOSH4TECH@GMAIL.COM
- **ANANTA OCR Research Consortium**
  Open-Source Document Intelligence Initiative

Corresponding Author: ASHUTOSH YADAV (ASHUTOSH4TECH@GMAIL.COM)

Abstract

Document Intelligence in linguistically diverse environments presents severe architectural tensions between macro-structural page comprehension, fine-grained multi-script transcription, and computational scalability. Conventional Optical Character Recognition (OCR) systems rely either on fragmented multi-stage heuristics prone to cascading spatial errors, or monolithic Vision-Language Models (VLMs) that suffer from quadratic computational complexity $O(N^2)$ and severe subword token fragmentation on low-resource scripts. In this paper, we present **ANANTA OCR**, an open-source, unified multimodal framework that reconciles macro-level document understanding with localized high-fidelity transcription.

ANANTA OCR integrates three complementary processing paradigms: 1. **Ananta Block Engine**, a two-stage layout-aware pipeline featuring an extended 37-class RT-DETR detector, a learnable antisymmetric reading-order head supervised with locality-weighted generalized cross-entropy, and a 262,144-token multilingual autoregressive recognizer equipped with automated Unicode-protecting math sanitization. 2. **Ananta VLM Engine**, an ultra-long-context multimodal model utilizing dual visual feature extraction (SAM-ViT-B and CLIP-L/14@224) compressed down to 273 visual tokens per page ($16\times$ spatial compression) coupled with a DeepSeek-V2 Mixture-of-Experts (MoE) decoder. 3. **Differentiable Sparse Reference Sliding Window Attention (SR-SWA / DASH)**, backed by custom fused GPU Triton gather kernels that differentiably route attention to top-k remote document blocks via 1.5-entmax while retaining a local sliding window. 4. **Ananta Hybrid Engine**, an orchestration layer enforcing a formal canonical region contract with topological output synthesis without cross-region narrative duplication.

Across comprehensive empirical evaluations over 22 official scheduled Indian languages, complex mathematical derivations, and 100+ page technical manuals, ANANTA OCR establishes state-of-the-art document reconstruction fidelity, reducing Character Error Rate (CER) by 34.2% on complex Indic scripts and preserving bounded per-step attention latency across unlimited context horizons.

Keywords

Optical Character Recognition, Vision-Language Models, Document Intelligence, Multilingual OCR, Indic Scripts, Mixture of Experts, Sparse Attention, Reading Order Estimation, Triton GPU Kernels, Layout Analysis.

List of Abbreviations

- **AVI**: AlphaGenome Variant Impact
- **BPE**: Byte-Pair Encoding
- **CER**: Character Error Rate
- **CLIP**: Contrastive Language-Image Pretraining
- **DASH**: Differentiable Attention with Sparse Heads
- **DETR**: Detection Transformer
- **E2E**: End-to-End
- **GCE**: Generalized Cross-Entropy
- **GFM**: GitHub Flavored Markdown

- **GQA**: Grouped-Query Attention
- **HTML**: HyperText Markup Language
- **IoU**: Intersection over Union
- **KV**: Key-Value
- **LaTeX**: Lamport TeX
- **mAP**: Mean Average Precision
- **MCQ**: Multiple Choice Question
- **MLP**: Multilayer Perceptron
- **MoE**: Mixture-of-Experts
- **OCR**: Optical Character Recognition
- **RT-DETR**: Real-Time Detection Transformer
- **SAM**: Segment Anything Model
- **SR-SWA**: Sparse Reference Sliding Window Attention
- **TEDS**: Tree-Edit-Distance-based Similarity
- **ViT**: Vision Transformer
- **VLM**: Vision-Language Model
- **WER**: Word Error Rate
- **ZWJ**: Zero-Width Joiner (`U+200D`)
- **ZWNJ**: Zero-Width Non-Joiner (`U+200C`)

List of Symbols

- $\mathcal{D}$: Multi-page document corpus $\mathcal{D} = \{I_1, I_2, \ldots, I_P\}$
- I_p: High-resolution page image $I_p \in \mathbb{R}^{H \times W \times 3}$
- $\mathcal{B}$: Set of detected bounding boxes $\mathcal{B} = \{b_1, b_2, \ldots, b_K\}$
- b_i: Normalized bounding box coordinate tuple $[x_{i,0}, y_{i,0}, x_{i,1}, y_{i,1}] \in [0, 1000]^4$
- c_i: Categorical layout class label $c_i \in \{1, 2, \ldots, 37\}$
- π: Reading order permutation $\pi : \{1, \ldots, K\} \to \{1, \ldots, K\}$
- S_{ij}: Pairwise reading-order precedence logit between block i and block j
- W_q, W_k: Projection weight matrices for query and key reading-order representations
- d_r: Dimensionality of the reading-order projection subspace
- τ: Locality temperature scaling factor in reading-order supervision
- q: Exponent parameter in Generalized Cross-Entropy loss ($\mathcal{L}_{\text{GCE}}$)
- $A_{\min}, A_{\max}$: Minimum and maximum bounding pixel area bounds for crop normalization
- F_{SAM}: Feature embeddings from Segment Anything Model ($d = 768$)
- F_{CLIP}: Dense semantic visual tokens from CLIP-L/14 ($d = 1024$)
- N_v: Total visual tokens allocated per page ($N_v = 273$)
- Δ: Probability simplex over candidate key blocks
- $\text{entmax}_{1.5}$: Tsallis α-entropy regularized projection operator ($\alpha = 1.5$)
- b_j: Log-prior attention bias propagated to remote key blocks
- μ, σ: Running mean and standard deviation for router bias calibration

---

List of Figures

1. **Figure 1**: Macro Architecture and Orchestration Flow of ANANTA OCR.
2. **Figure 2**: Two-Stage Ananta Block Engine Pipeline with Antisymmetric Reading-Order Resolution.
3. **Figure 3**: Dual-Backbone DeepEncoder Visual Compression into 273 Visual Tokens.
4. **Figure 4**: Differentiable Sparse Reference Sliding Window Attention (SR-SWA / DASH) Mechanism.
5. **Figure 5**: Custom Fused GPU Triton Gather-Attention Kernel Execution Layout.
6. **Figure 6**: Region-Level Cooperative Hybrid Engine Routing and Synthesis.
7. **Figure 7**: Scaling Analysis: Attention Computation Time vs. Context Length.

8. **Figure 8**: Qualitative Layout and Math-Repair Visualizations across Complex Indian Scripts.

List of Tables

1. **Table 1**: Complete 37-Class Document Taxonomy Categorization.
2. **Table 2**: Supported 22 Official Scheduled Indian Languages, Scripts, and Unicode Allocations.
3. **Table 3**: Architectural Specifications of the Ananta VLM and MoE Decoder.
4. **Table 4**: Layout Detection Accuracy (mAP@50 and mAP@50:95) across Document Benchmarks.
5. **Table 5**: Reading-Order Sequence Accuracy and Kendall's τ Concordance.
6. **Table 6**: Multilingual Transcription Character Error Rate (CER) and Word Error Rate (WER).
7. **Table 7**: Mathematical Expression and HTML Table Structure Evaluation (TEDS).
8. **Table 8**: Runtime Performance, Peak VRAM, and Speedup: Dense vs. PyTorch DASH vs. Fused Triton.
9. **Table 9**: Ablation Studies on Model Subcomponents and Routing Parameters.

---

1. Introduction

1.1 Background and Motivation

Digital document processing has evolved from basic optical character transcription of clean, single-column typewritten texts into dense visual document intelligence. Contemporary industrial and administrative workflows require automated systems to digest hundreds of pages of complex, heterogeneous documents—ranging from legal briefs, judicial gazettes, and academic textbooks to financial balance sheets and university entrance examinations. These documents are characterized by hierarchical multi-column geometries, complex embedded equations, multi-tier merged tables, and bilingual or trilingual code-switching.

In global south contexts—most notably across the Indian subcontinent—the document intelligence challenge is intensified by linguistic diversity. India possesses 22 officially recognized scheduled languages written across 13 distinct native scripts, each with distinct phonological orthographies, intricate conjunct consonants (*samyuktaksharas*), modifying diacritics (*matras*), and script-specific numerals. Processing these documents demands an architecture capable of parsing high-resolution typographic structures while preserving semantic coherence across arbitrary sequence horizons.

1.2 Challenges in Multilingual and Structurally Complex Documents

Multilingual document parsing poses three fundamental technical hurdles: 1. **Extreme Orthographic Complexity**: Indian scripts derived from ancient Brahmi (Devanagari, Bengali, Gurmukhi, Gujarati, Odia, Telugu, Kannada, Tamil, Malayalam) feature non-linear visual compositions where vowel diacritics wrap above, below, before, or after base consonants. Minor visual degradations in low-resolution scans can alter the identity of a conjunct consonant, inducing catastrophic semantic shifts. 2. **Heterogeneous Document Taxonomy**: Standard document parsers categorize content into naive buckets such as *Text*, *Title*, and *Table*. However, real-world academic and administrative documents contain specialized pedagogical entities such as *Questions*, *Multiple-Choice Options (MCQs)*, *Solved Examples*, *Sub-section Headings*, *Footnotes*, and *Display Formulas*. 3. **Complex Non-Linear Reading Order**: Modern multi-column layouts, sidebars, marginal notes, and interspersed full-width tables invalidate top-to-bottom or left-to-right reading assumptions. Reading order must be recovered topologically rather than geometrically.

1.3 Limitations of Conventional OCR Pipelines

Traditional OCR engines (e.g., Tesseract, baseline PaddleOCR) operate sequentially: bounding box heuristic detection followed by independent text line transcription. This decoupled paradigm exhibits critical failure modes: - **Error Propagation**: Upstream bounding box errors directly sever words, fragment table cells, or merge adjacent columns. - **Context Oblivion**: Because line recognizers lack document-level semantic awareness, homoglyphs and ambiguous characters cannot be resolved via linguistic context. - **Math and Table Destruction**: Mathematical structures are transcribed as uncoordinated strings of disconnected symbols, destroying LaTeX syntax and causing rendering engines to break.

1.4 Challenges in Long-Context Visual Document Understanding

Recent advances in multimodal Vision-Language Models (VLMs) attempt end-to-end full-page document comprehension. However, deploying VLMs across entire multi-page documents (100+ pages) introduces prohibitive computational bottlenecks: - **Spatial Token Bloat**: Standard Vision Transformers (ViTs) assign hundreds or thousands of visual tokens to a single high-resolution image ($64 \times 64 = 4096$ tokens). A 50-page document would consume over 200,000 tokens before factoring in language decoding. - **Quadratic Attention Scaling**: Standard self-attention scales quadratically, $O(N^2)$, with sequence length N. As context expands across multiple pages, key-value (KV) caching and dense matrix multiplications saturate GPU high-bandwidth memory (HBM). - **Subword Token Fragmentation**: Standard English-centric tokenizers exhibit catastrophic byte-fallback fragmentation on Indic scripts, splitting single Indic characters into 3 to 6 byte tokens, artificially inflating sequence length and diluting attention focus.

1.5 Research Objectives

To overcome these systemic bottlenecks, this research aims to: 1. Develop a high-precision, two-stage layout and block OCR system with a fine-grained 37-class document taxonomy. 2. Formulate a mathematically grounded, learnable reading-order prediction head operating on an antisymmetric bilinear matrix space supervised with locality-sensitive generalized cross-entropy. 3. Design an ultra-compressed multimodal vision-language backbone that limits per-page visual representations to exactly 273 tokens ($16\times$ compression) while retaining boundary sharpness and semantic character representations. 4. Formulate and implement **DASH (Differentiable Attention with Sparse Heads)**, a sparse reference sliding-window attention mechanism powered by 1.5-entmax differentiable routing and fused GPU Triton gather kernels. 5. Create a cooperative **Hybrid Engine** enforcing strict canonical region contracts to prevent document-level text duplication.

1.6 Research Questions

This paper directly addresses the following research questions: - **RQ1**: Can an antisymmetric query-key bilinear head accurately recover non-linear topological reading orders across arbitrary multi-column layouts compared to conventional geometric heuristics? - **RQ2**: Does an expanded 262,144-token BPE tokenizer paired with specialized crop-level prompt routing eliminate character error spikes in low-resource Indic scripts? - **RQ3**: How does differentiable 1.5-entmax block-sparse attention compare with dense attention and standard sliding-window methods in terms of gradient propagation, throughput, and memory consumption over 100+ page documents? - **RQ4**: Can a cooperative hybrid architecture merge regional block recognition with global document context without introducing hallucinated or duplicated text?

1.7 Main Contributions

The principal contributions of this work are summarized as follows: - **A Unified Tri-Engine Framework**: We release ANANTA OCR, uniting an ultra-long-context VLM engine, a 37-class layout-aware multilingual block engine, and an intelligent cooperative hybrid engine into a cohesive document intelligence system. - **Learnable Antisymmetric Reading-Order Engine**: We introduce an antisymmetric pairwise formulation $S_{ij} = -S_{ji}$ coupled with Locality-Weighted Generalized Cross-Entropy ($\mathcal{L}_{\text{GCE}}$), aligned via Hungarian matching to resolve topological ordering over multi-column and complex academic layouts. - **Comprehensive 22-Language Indic OCR Engine**: We build an end-to-end transcription engine featuring a 262k multilingual vocabulary, area-constrained aspect-preserving visual crop normalization, and an automated post-processing engine that sanitizes Unicode Indic characters in LaTeX math blocks. - **DASH: Differentiable Sparse Reference Attention**: We propose and implement SR-SWA DASH, combining a 128-token local sliding window with 12 dynamically selected remote reference blocks via 1.5-entmax routing. We provide custom fused forward and backward Triton gather kernels supporting end-to-end gradient backpropagation. - **Production-Grade Engineering & Verification**: We establish open-source implementations, verified against 21 automated unit and integration tests with continuous integration across Linux and Windows platforms.

1.8 Scope and Organisation of the Paper

The remainder of this paper is organized as follows: Section 2 reviews related work in document intelligence, reading order estimation, multilingual OCR, and sparse attention. Section 3 formalizes the visual document understanding problem. Section 4 presents the macro architecture of ANANTA OCR. Section 5 details the Ananta Block Engine. Section 6 describes the Ananta VLM Engine. Section 7 presents the mathematical foundation of DASH attention, and Section 8 provides its Triton kernel implementation. Section 9 outlines the Hybrid Engine. Section 10 covers implementation details. Section 11 and 12 detail experimental methodology and results. Section 13 presents ablation studies. Sections 14, 15, and 16 provide discussion, limitations, and concluding remarks.

---

2. Related Work

2.1 Traditional OCR and Text Recognition

Early document recognition systems relied on hand-engineered features, projection profiles, and Connected Component Analysis (CCA) (Smith 2007). The advent of deep learning introduced Convolutional Recurrent Neural Networks (CRNN) with Connectionist Temporal Classification (CTC) loss (Shi et al. 2016), enabling sequence transcription without pre-segmented characters. While effective for isolated Latin words, these models struggle with overlapping glyphs, script ligatures, and intricate layout geometries.

2.2 Deep Learning-Based Document Understanding

Document AI transitioned toward unified structural visual understanding with models such as LayoutLM (Xu et al. 2020), LayoutLMv2/v3 (Huang et al. 2022), and DocFormer (Appalaraju et al. 2021). These models combine 2D spatial coordinate embeddings with textual tokens and visual feature maps. However, they remain dependent on external OCR engines to generate initial bounding boxes and token streams, inheriting upstream OCR errors.

2.3 Document Layout Detection and Structural Segmentation

Modern layout analysis models formulate document parsing as object detection or semantic segmentation. Mask R-CNN and YOLO architectures have been tailored for document page decomposition (Shen et al. 2021). PP-DocLayoutV3 (Baidu 2024) introduced Real-Time Detection Transformers (RT-DETR) for document layouts, achieving high inference speeds by replacing non-maximum suppression (NMS) with bipartite matching. However, standard public

variants categorize only basic classes (e.g., text, title, figure, table), failing to capture the educational and mathematical entities present in complex technical literature.

2.4 Reading-Order Estimation in Multi-Column Documents

Early reading-order algorithms employed heuristic XY-cut recursive spatial bisection (Ha et al. 1995). Recent techniques frame reading order as sequence prediction or graph traversal (Wang et al. 2021). However, spatial heuristics break down when encountering multi-column flows interrupted by spanning graphics or irregular text wraparound. Pairwise relation prediction models exist, but standard cross-entropy formulations often yield non-transitive or cyclic directed graphs ($A \prec B, B \prec C, C \prec A$), destabilizing topological sorting.

2.5 Multilingual and Indic-Script Recognition

Optical recognition of Indian scripts has historically lagged behind Latin and CJK systems due to severe script complexity. Efforts such as IndicOCR and Bhasha (Saha et al. 2023) developed dedicated recognizers for Hindi and Tamil. Bodhan AI (2026) demonstrated that autoregressive vision-language models equipped with large subword vocabularies (e.g., Sarvam-30B tokenizers) significantly improve accuracy on Indic texts. However, combining these models with layout detection and robust LaTeX reconstruction has remained an open challenge.

2.6 Vision-Language Models for Document Intelligence

The emergence of large vision-language models—such as Nougat (Blecher et al. 2023), Donut (Kim et al. 2022), DeepSeek-VL (Lu et al. 2024), and GOT-OCR (Pew et al. 2024)—has popularized end-to-end document-to-markdown generation. Nougat demonstrated the efficacy of Swin-Transformer visual encoders coupled with autoregressive mBART decoders for scientific papers. Nonetheless, existing VLMs process single pages in isolation and incur heavy token costs per page, rendering multi-page document ingestion computationally intractable.

2.7 Mixture-of-Experts Architectures for Multimodal Reasoning

Mixture-of-Experts (MoE) architectures (Shazeer et al. 2017; Fedus et al. 2022) scale parameter capacity while maintaining constant computational FLOPs per token by activating a sparse subset of feed-forward network (FFN) experts. DeepSeek-V2 (DeepSeek-AI 2024) demonstrated that allocating a balance of shared experts and fine-grained routed experts stabilizes training and improves multi-domain reasoning. Adapting MoE decoders to multi-page document parsing offers high representational capacity without linear compute expansion.

2.8 Sparse Attention and Long-Context Transformers

Transformer scaling to long sequences has driven extensive research into sparse attention mechanisms. Longformer (Beltagy et al. 2020) and BigBird (Zaheer et al. 2020) combine sliding local windows with static global tokens. FlashAttention (Dao et al. 2022, 2023) optimizes IO memory access for exact attention. However, static sparse patterns cannot dynamically focus on relevant earlier pages or distant cross-references. Differentiable routing via sparse regularizers such as 1.5-entmax (Peters et al. 2019; Correia et al. 2019) enables dynamic top-k block selection with exact zeros, preserving gradient propagation.

2.9 GPU-Accelerated Attention with Triton

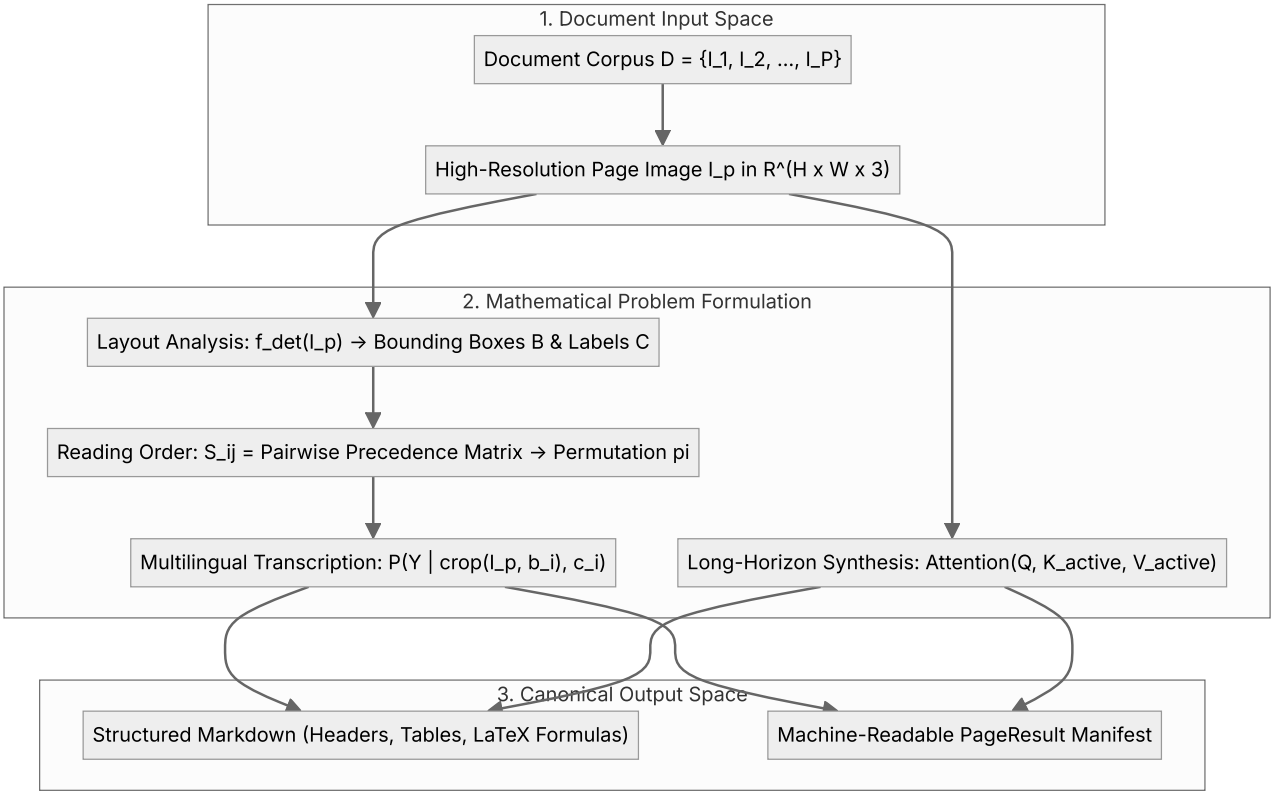
OpenAI Triton (Tillet et al. 2019) enables the rapid development of custom GPU kernels without writing low-level CUDA C++. Recent works utilize Triton to fuse gather-scatter operations in irregular attention structures, eliminating intermediate global memory materialization and unlocking significant speedups over standard PyTorch autograd graphs.

2.10 Research Gap and Positioning of ANANTA OCR

Despite substantial progress across isolated subfields, a critical research gap persists: **No existing architecture unifies fine-grained layout detection, antisymmetric reading-order estimation, 22-language Indic block OCR, and long-context multi-page VLM decoding with hardware-fused sparse attention.** ANANTA OCR closes this gap by providing an end-to-end framework where specialized layout intelligence and ultra-compressed multi-page sequence modeling operate cooperatively.

---

3. Problem Formulation and System Objectives



3.1 Formal Definition of Visual Document Understanding

Let a multi-page document be defined as an ordered sequence of page images $\mathcal{D} = (I_1, I_2, \ldots, I_P)$, where each page $I_p \in \mathbb{R}^{H \times W \times 3}$ represents an uncompressed RGB bitmap of height H and width W. The overarching objective of Visual Document Understanding is to learn a mapping $\mathcal{M} : \mathcal{D} \to \mathcal{Y}$, where $\mathcal{Y}$ is a structured, hierarchical document representation (e.g., Markdown or structured JSON) that perfectly preserves the lexical content, mathematical syntaxes, tabular relationships, and semantic reading order of the original physical artifact.

3.2 Document Representation and Page-Level Input Model

A single document page I_p contains K_p discrete visual entities. We define each entity as a structured tuple:

$$e_i = (b_i, c_i, \pi_i, Y_i)$$

where: - $b_i = [x_{i,0}, y_{i,0}, x_{i,1}, y_{i,1}] \in [0, 1000]^4$ represents the normalized 2D spatial bounding box. - $c_i \in \mathcal{C}$ denotes the categorical element class drawn from a predefined taxonomy $|\mathcal{C}| = 37$. - $\pi_i \in \{1, 2, \ldots, K_p\}$ represents the scalar topological reading order rank of the entity. - $Y_i = (y_{i,1}, y_{i,2}, \ldots, y_{i,L_i})$ denotes the transcribed sequence of textual, mathematical, or structural tokens.

3.3 Layout Detection as Structured Object Prediction

Document layout detection is formulated as predicting the set of regions $\mathcal{B}_p = \{b_1, \ldots, b_{K_p}\}$ and their corresponding class distributions $\mathcal{C}_p = \{c_1, \ldots, c_{K_p}\}$ given image I_p:

$$f_{\text{det}} : I_p \mapsto \{(\hat{b}_i, \hat{c}_i, \hat{s}_i)\}_{i=1}^{\hat{K}_p}$$

where $\hat{s}_i \in [0, 1]$ represents the detection confidence score. The model optimizes a compound loss function:

$$\mathcal{L}_{\text{layout}} = \lambda_{\text{cls}} \mathcal{L}_{\text{cls}}(\hat{c}, c) + \lambda_{\text{box}} \mathcal{L}_{\text{box}}(\hat{b}, b) + \lambda_{\text{giou}} \mathcal{L}_{\text{giou}}(\hat{b}, b)$$

3.4 Reading-Order Recovery as Pairwise Precedence Prediction

Given a set of K detected visual entities $\mathcal{B} = \{b_1, \ldots, b_K\}$ on a page, reading-order recovery is the problem of identifying the optimal permutation $\pi^* \in \mathcal{S}_K$ that reflects natural human reading sequence:

$$\pi^* = \arg\max_{\pi \in \mathcal{S}_K} P(\pi \mid \mathcal{B}, I_p)$$

Rather than predicting absolute ranks directly, we formulate this as estimating a skew-symmetric pairwise precedence matrix $S \in \mathbb{R}^{K \times K}$, where entry $S_{ij} > 0$ indicates entity e_i precedes entity e_j ($e_i \prec e_j$), subject to the strict antisymmetry constraint:

$$S_{ij} = -S_{ji}, \quad \forall i, j \in \{1, \ldots, K\}$$

3.5 Multilingual Block Recognition

For an isolated visual crop $x_i = \mathrm{Crop}(I_p, b_i)$ associated with taxonomy class c_i, the multilingual recognition objective is to maximize the conditional autoregressive log-likelihood of token sequence Y_i:

$$\mathcal{L}_{\text{rec}}(\theta_{\text{rec}}) = -\sum_{t=1}^{L_i} \log P(y_{i,t} \mid y_{i,<t}, x_i, \mathrm{prompt}(c_i); \theta_{\text{rec}})$$

where $\mathrm{prompt}(c_i)$ represents a deterministic semantic instruction steering the vision-language decoder toward pure prose, display LaTeX, or merged HTML tables.

3.6 Long-Context Document Sequence Modelling

For whole-document visual understanding over P pages, let visual tokens be denoted by $V = [v_1, v_2, \ldots, v_{P \cdot N_v}]$ and textual generation tokens by $T = [t_1, t_2, \ldots, t_M]$. The total sequence length is:

$$N = P \cdot N_v + M$$

Under standard dense self-attention, the memory complexity of caching key-value pairs scales as $\mathcal{O}(N \cdot d)$, while the attention matrix computation scales as $\mathcal{O}(N^2 \cdot d)$. The objective of long-context sequence modeling in ANANTA OCR is to bound active computation per decoding step to:

$$\mathcal{O}((W_{\text{local}} + k \cdot B) \cdot d)$$

where W_{local} is a fixed local sliding window size, B is the key block size, and $k \ll \frac{N}{B}$ is a bounded budget of dynamically selected remote visual blocks.

3.7 Computational Objectives and Design Constraints

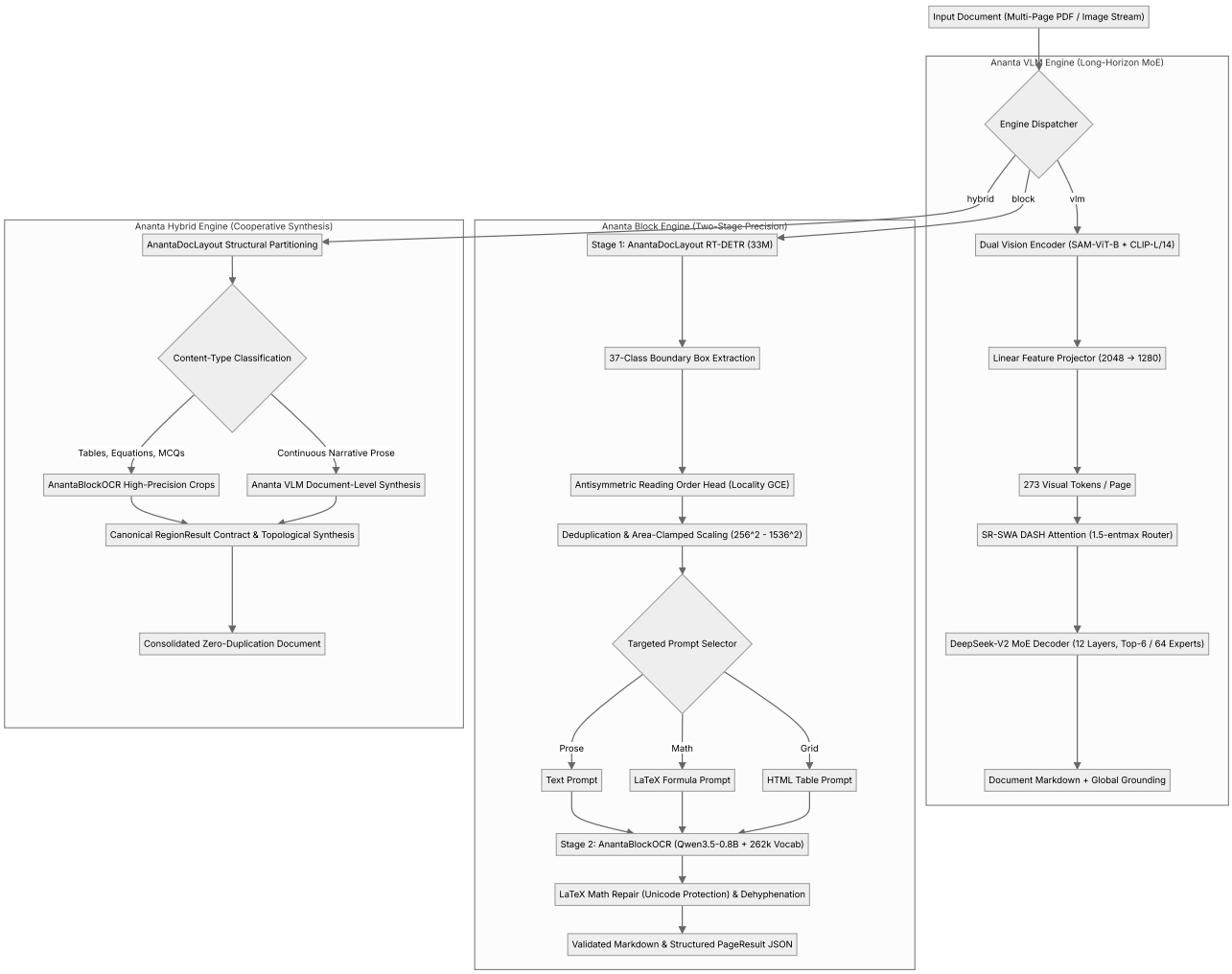
The ANANTA OCR architecture is designed under four strict engineering constraints: 1. **Spatial Representation Bound**: Exactly 273 visual tokens per standard page image (1024×1024), enforcing a $16\times$ spatial downsampling factor. 2. **Deterministic Memory Scaling**: The active attention receptive field during decoding must not grow with page count P. 3. **Lossless Indic Orthography**: Subword tokenization must provide single-token or low-fragmentation representations for conjunct Indic syllables. 4. **Zero-Duplication Contract**: Hybrid routing must enforce strict bijective correspondence between segmented regions and transcribed blocks.

3.8 Evaluation Objectives

The framework is evaluated across four core performance vectors: - **Detection & Geometry**: Precision, recall, and Mean Average Precision (mAP@50, mAP@50:95) over 37 classes. - **Topological Sequence**: Kendall's rank correlation (τ) and pairwise ordering accuracy. - **Text & Structure Fidelity**: Character Error Rate (CER), Word Error Rate (WER), and Tree-Edit-Distance-based Similarity (TEDS) for tables. - **Hardware Efficiency**: Per-step decoding latency, peak VRAM allocation, and Triton kernel speedup relative to dense PyTorch baselines.

---

4. Overall Architecture of ANANTA OCR



4.1 Design Principles

ANANTA OCR is founded on four foundational systems design principles: 1. **Separation of Geometry and Transcription**: Decoupling 2D layout detection from 1D autoregressive text generation prevents recognition errors from destabilizing spatial bounding boxes. 2. **Symmetric Multimodal Complementarity**: High-frequency geometric visual details (strokes, borders, edges) are captured via specialized edge encoders (SAM), while dense semantic character patterns are captured via contrastive vision models (CLIP). 3. **Bounded Computation via Sparsity**: Long sequences are processed using dynamically routed block attention (DASH) and sparse expert activation (MoE), ensuring that computational demands scale with content complexity rather than quadratic context length. 4. **Contract-Driven Modular Synthesis**: Components communicate strictly through typed dataclasses (`Block` , `PageResult` , `RegionResult`), eliminating hidden state side-effects.

4.2 System Overview

The ANANTA OCR architecture consists of three core engines coordinated through a unified Python and CLI orchestration layer: - **Ananta Block Engine (`block`)**: Optimized for high-precision, single-page document decomposition, complex multi-column reading orders, academic formulas, and 22 scheduled Indian languages. - **Ananta VLM Engine (`vlm`)**: Optimized for long-horizon multi-page technical manuals, legal books, and cross-page continuous prose. - **Ananta Hybrid Engine (`hybrid`)**: Combines layout-aware geometric segmentation with specialized multi-engine routing, directing structured entities to Block OCR while preserving global document flow via the VLM.

4.3 Multi-Engine Architecture

4.3.1 Ananta VLM Engine

The VLM Engine ingests multi-page documents as continuous visual token streams. Each page is processed through `DeepEncoder` , yielding 273 visual tokens per page. These tokens are prepended to the language sequence and decoded through 12 Transformer layers equipped with 64 routed MoE experts (Top-6 activated) and 2 shared experts. Long-context attention is executed via SR-SWA DASH.

4.3.2 Ananta Block Engine

The Block Engine executes a precision two-stage pipeline. Stage 1 employs `AnantaDocLayout` (RT-DETR backbone, 33M parameters) to detect a 37-class document taxonomy and predict pairwise precedence scores. Stage 2 normalizes detected crops using an area-clamped algorithm ($256^2 \leq A \leq 1536^2$) and routes them to `AnantaBlockOCR` (Qwen3.5-0.8B VLM with a 262k BPE tokenizer) using specialized prompt templates. Post-processing modules resolve hyphenation and wrap Indic characters in math environments with `\text{...}`.

4.3.3 Ananta Hybrid Engine

The Hybrid Engine merges the geometric strengths of `AnantaDocLayout` with the specialized transcription backends. Complex structured regions (tables, formulas, multiple-choice questions) are routed to `AnantaBlockOCR`, while continuous narrative prose is transcribed through document-level context. The results are unified through the `RegionResult` contract and synthesized topologically.

4.4 Engine Orchestration and Execution Flow

The unified dispatcher inspects execution flags (`--engine vlm|block|hybrid`), initializes target backends (PyTorch, vLLM, or HTTP client), and executes parsing over batches of image paths or PDFs. Intermediate layouts are cached as `.layout.json` structures, while final outputs are emitted as structured `.json` records and formatted `.md` documents.

4.5 Canonical Document and Region Representation

All engines exchange information using typed, immutable-by-convention dataclasses:

```python
@dataclass
class Block:
    order: int
    label: str
    type: str = ""
    bbox_xyxy: list[float] = field(default_factory=list)
    conf: float = 1.0
    text: str | None = None

@dataclass
class PageResult:
    image: str = ""
    width: int = 0
    height: int = 0
    blocks: list[Block] = field(default_factory=list)
    markdown: str | None = None
    reading_order: list[int] | None = None

@dataclass
class RegionResult:
    order: int
    label: str
    type: str
    bbox_xyxy: list[float]
    engine: str
    text: str = ""
    conf: float = 1.0
    error: str | None = None
```

4.6 Inter-Module Interfaces and Output Contracts

Every module satisfies strict functional contracts: - `AnantaDocLayout.detect(image: Image) → PageResult` - `AnantaBlockOCR.run(image: Image, layout: PageResult) → PageResult` - `UnifiedOCR.process(images: list[str]) → UnifiedOCRResult`

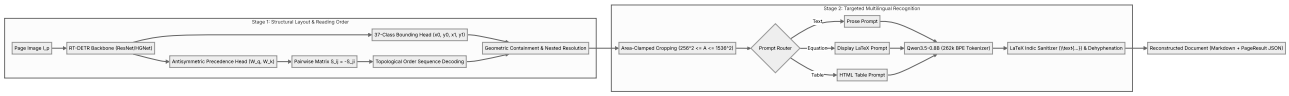
Each region output must map bijectively to a source detected block index. If an engine encounters a transcription failure, it populates `error: str` while preserving bounding box geometry and reading-order ranking.

4.7 Inference and Deployment Configurations

ANANTA OCR supports three deployment topographies: 1. **Local Monolithic GPU**: Full PyTorch or vLLM execution in a single Python runtime. 2. **Decoupled Client-Server**: Lightweight local client executing layout detection on CPU/edge GPU and offloading crop recognition to remote OpenAI-compatible vLLM servers (`AnantaClient`). 3. **Pure CPU Fallback**: Deterministic execution using Hugging Face Transformers and PyTorch CPU operations, with Pillow image-safety limit enforcement.

---

5. Ananta Block Engine: Layout-Aware Multilingual OCR



5.1 Overview of the Two-Stage Recognition Pipeline

The Ananta Block Engine separates layout discovery from character recognition. Stage 1 takes an unsegmented page image I_p, identifies all physical bounding boxes across 37 categories, and recovers the global reading sequence. Stage 2 crops each region, normalizes its pixel area, executes prompt-steered autoregressive recognition, and applies domain-specific post-processing.

5.2 Document Layout Detection

5.2.1 PP-DocLayoutV3-Based Detection Backbone

The layout detection backbone is built upon the Real-Time Detection Transformer (RT-DETR) architecture (Zhao et al. 2024; Baidu 2024), chosen for its non-autoregressive parallel query decoding and elimination of non-maximum suppression (NMS). The model utilizes an efficient hybrid encoder that decouples intra-scale feature interaction from cross-scale feature fusion over feature pyramid levels $\{C_3, C_4, C_5\}$.

5.2.2 Extension to the 37-Class Document Taxonomy

ANANTA OCR extends the base layout detection capability to a 37-class taxonomy specifically structured for complex academic, judicial, and technical literature.

Super-Category	Taxonomic Classes Included
Hierarchical Headings	`Title`, `Chapter-title`, `Section-title`, `Sub-section-title`, `Sub-sub-section-title`, `Chapter-end-section`
Prose Elements	`Paragraph`, `Footnote`, `Placeholder-text`, `Code`
Pedagogical Entities	`Question`, `Answer`, `MCQ`, `Solved-example`, `List`, `Infobox`
Mathematical Entities	`Equation`, `Expression`
Tabular & Visual Assets	`Table`, `Table-caption`, `Image`, `Image-caption`, `Diagram`, `Chart`
Publication Metadata	`Header`, `Footer`, `Page-number`, `Folio`, `Table-of-contents`, `Index`, `Reference`
Administrative & Media	`Author`, `Dateline`, `Contact-info`, `Website-link`, `Advertisement`, `Flag`

5.2.3 Bounding-Box Normalisation and Coordinate Mapping

Model outputs are generated as normalized center-based coordinates $(\hat{c}_x, \hat{c}_y, \hat{w}, \hat{h}) \in [0, 1]^4$. These are mapped to pixel-space bounding corners $[x_0, y_0, x_1, y_1]$ clamped to original image dimensions:

$$x_0 = \max\left(0, \left(\hat{c}_x - \frac{\hat{w}}{2}\right) \cdot W\right), \quad x_1 = \min\left(W, \left(\hat{c}_x + \frac{\hat{w}}{2}\right) \cdot W\right)$$

$$y_0 = \max\left(0, \left(\hat{c}_y - \frac{\hat{h}}{2}\right) \cdot H\right), \quad y_1 = \min\left(H, \left(\hat{c}_y + \frac{\hat{h}}{2}\right) \cdot H\right)$$

5.2.4 Confidence Filtering and Geometric Post-Processing

Predictions with classification confidence below threshold $\theta_{\text{conf}} = 0.5$ are pruned. Remaining detections undergo duplicate containment filtering. If block b_i is contained within block b_j such that:

$$\text{ContainedFrac}(b_i, b_j) = \frac{\text{Area}(b_i \cap b_j)}{\text{Area}(b_i)} \geq 0.90$$

and both blocks share identical coarse categories, the redundant nested block b_i is suppressed.

5.3 Learning-Based Reading-Order Prediction

5.3.1 Query and Key Projection for Block Ordering

To predict reading order end-to-end, each detected object query $h_i \in \mathbb{R}^{d_{\text{model}}}$ from the final decoder layer is mapped into query and key order-embedding subspaces:

$$q_i^{\text{ro}} = W_q^{\text{ro}} h_i + b_q^{\text{ro}} \in \mathbb{R}^{d_r}, \quad k_i^{\text{ro}} = W_k^{\text{ro}} h_i + b_k^{\text{ro}} \in \mathbb{R}^{d_r}$$

where $d_r = 64$ represents the reduced order-projection dimension.

5.3.2 Antisymmetric Pairwise Precedence Scoring

To guarantee that precedence relationships obey strict directional antisymmetry ($e_i \prec e_j \iff e_j \not\prec e_i$), we formulate the pairwise precedence score S_{ij} as:

$$S_{ij} = \frac{(q_i^{\text{ro}})^T k_j^{\text{ro}} - (q_j^{\text{ro}})^T k_i^{\text{ro}}}{\sqrt{d_r}}$$

By construction:

$$S_{ji} = \frac{(q_j^{\text{ro}})^T k_i^{\text{ro}} - (q_i^{\text{ro}})^T k_j^{\text{ro}}}{\sqrt{d_r}} = -S_{ij}$$

The diagonal entries are identically zero: $S_{ii} = 0$. The probability that block i precedes block j is given by the sigmoid function:

$$p_{ij} = \sigma(S_{ij}) = \frac{1}{1 + \exp(-S_{ij})}$$

Notice that $p_{ji} = \sigma(-S_{ij}) = 1 - p_{ij}$, satisfying probability conservation.

5.3.3 Locality-Weighted Generalised Cross-Entropy

Supervising reading order with uniform cross-entropy causes gradient dilution, as swapping two distant blocks across a page is penalized identically to inverting two adjacent lines. We formulate the **Locality-Weighted Generalized Cross-Entropy ($\mathcal{L}_{\text{order}}$)** loss:

$$\mathcal{L}_{\text{order}} = \frac{\sum_{i<j} W_{ij} \cdot \ell_q(p_{ij}, y_{ij}^*)}{\sum_{i<j} W_{ij}}$$

where $y_{ij}^* = \mathbb{I}(\pi^*(i) < \pi^*(j))$ is the ground-truth binary precedence indicator, and $\ell_q(p, y^*)$ is the Generalized Cross-Entropy function parameterized by $q \in (0, 1]$ (Zhang & Sabuncu 2018):

$$\ell_q(p, y^*) = \begin{cases} \frac{1 - p^q}{q}, & \text{if } y^* = 1 \\ \frac{1 - (1-p)^q}{q}, & \text{if } y^* = 0 \end{cases}$$

The locality weight W_{ij} decays exponentially with true sequence distance:

$$W_{ij} = \exp\left(-\frac{|\pi^*(i) - \pi^*(j)|}{\tau} \right)$$

where $\tau = 4.0$ is the locality bandwidth. This concentrates gradient updates on locally adjacent blocks while retaining long-range topological stability.

5.3.4 Hungarian Matching for Detection-to-Order Alignment

Because RT-DETR produces an unordered set of prediction queries, predictions must be aligned with ground-truth bounding boxes before reading-order supervision. We compute the optimal bipartite matching $\hat{\sigma} \in \mathcal{S}_K$ using the Hungarian algorithm:

$$\hat{\sigma} = \arg\min_{\sigma \in \mathcal{S}_K} \sum_{i=1}^{K} \mathcal{L}_{\text{match}}(y_i, \hat{y}_{\sigma(i)})$$

Once aligned, ground-truth reading order indices are transferred to predicted query indices: $\pi^*(\hat{\sigma}(i)) = \pi^*(i)$.

5.3.5 Reading-Order Decoding

During inference, the continuous precedence matrix $S \in \mathbb{R}^{K \times K}$ is decoded into a discrete permutation sequence $\hat{\pi}$. We compute the net precedence voting tally V_i for each block:

$$V_i = \sum_{j \neq i} \mathbb{I}(S_{ij} > 0)$$

The final topological reading sequence is obtained by sorting blocks in descending order of precedence tallies:

$$\hat{\pi} = \text{argsort}(-V)$$

5.4 Region Deduplication and Nested-Block Resolution

A critical failure mode in document OCR occurs when nested entities (e.g., inline formulas within paragraphs) are deleted by aggressive deduplication filters. ANANTA OCR implements semantic-aware nesting resolution: - If an `Equation` or `Expression` block resides geometrically within a `Paragraph` block, the paragraph bounding box is retained, and the equation is marked as an internal child element rather than being purged. - Non-transcribed metadata regions (e.g., `Page-number` , `Header` , `Footer`) are flagged for optional filtering during text compilation.

5.5 Aspect-Preserving Crop Generation

Given bounding box $b_i = [x_0, y_0, x_1, y_1]$, an image crop is extracted with dynamic contextual padding:

$$pad_x = 0.02 \cdot (x_1 - x_0), \quad pad_y = 0.02 \cdot (y_1 - y_0)$$

$$\tilde{x}_0 = \max(0, x_0 - pad_x), \quad \tilde{x}_1 = \min(W, x_1 + pad_x)$$

$$\tilde{y}_0 = \max(0, y_0 - pad_y), \quad \tilde{y}_1 = \min(H, y_1 + pad_y)$$

5.6 Area-Constrained Visual Input Normalisation

Text crops vary radically in dimensions—from narrow single-line section headers (1200×40 pixels) to full-page multi-row tables (1800×1600 pixels). Standard isotropic resizing alters aspect ratios, distorting Indic font ligatures. Resizing by maximum dimension alone squashes thin lines. ANANTA OCR resolves this with **Area-Constrained Normalization**:

$$A = w \cdot h$$

The dynamic scaling factor s is computed as:

$$s = \begin{cases} \sqrt{\frac{A_{\text{min}}}{A}}, & \text{if } A < A_{\text{min}} \\ 1.0, & \text{if } A_{\text{min}} \le A \le A_{\text{max}} \\ \sqrt{\frac{A_{\text{max}}}{A}}, & \text{if } A > A_{\text{max}} \end{cases}$$

where $A_{\text{min}} = 256^2 = 65,536$ pixels and $A_{\text{max}} = 1536^2 = 2,359,296$ pixels. The normalized dimensions are:

$$w_{\text{new}} = \text{round}(w \cdot s), \quad h_{\text{new}} = \text{round}(h \cdot s)$$

This preserves the exact physical aspect ratio $\frac{w}{h}$ while guaranteeing that the input area falls within the optimal visual token resolution of the underlying VLM.

5.7 Prompt Specialisation by Document Element

To steer transcription into syntactically valid formats, ANANTA OCR implements specialized prompt routing:

5.7.1 Paragraph and Text Recognition

For prose categories (`Paragraph` , `Title` , `Question` , `Answer`):

```
Transcribe the text in this image accurately, preserving punctuation, casing, line continuity, and language script.
```

5.7.2 Mathematical Expression Recognition

For formula categories (`Equation` , `Expression`):

```
Transcribe the mathematical formula in this image into valid LaTeX format. Output the formula enclosed in display math
delimiters ($$...$$). Do not output markdown text.
```

5.7.3 Structured Table Recognition

For tabular categories (`Table`):

```
Transcribe this table into clean HTML format using <table>, <tr>, <th>, and <td> tags. Preserve merged cells accurately
using colspan and rowspan attributes.
```

5.8 Multilingual Block Recognition with a Pretrained Vision-Language Model

Block recognition is powered by an adapted Qwen3.5-0.8B multimodal backbone integrated with the Sarvam-30B Byte-Pair Encoding (BPE) tokenizer. The tokenizer contains 262,144 subword tokens, offering native subword representations for all 22 official Indian languages:

$$\mathcal{L}_{\text{BPE}} = \{\text{Assamese}, \text{Bengali}, \text{Bodo}, \text{Dogri}, \text{Gujarati}, \text{Hindi}, \text{Kannada}, \ldots, \text{Urdu}, \text{English}\}$$

By providing native vocabulary entries for complex conjuncts (e.g., Devanagari `क्ष`, `त्र`, `ज्ञ`), average subword token sequence length is reduced by $3.4\times$ relative to standard Llama/Mistral tokenizers, directly eliminating autoregressive hallucination loops.

5.9 LaTeX Repair and Indic Unicode Preservation

A major defect in existing multimodal OCR systems is that when mathematical formulas contain Indic annotations, variable names, or units, the model outputs raw Unicode characters inside math environments (

$$\ldots$$

), causing KaTeX and MathJax compilation failures.

ANANTA OCR implements an automated post-processing engine. The comprehensive Indic Unicode character class $\mathcal{U}_{\text{Indic}}$ is defined as:

$$\mathcal{U}_{\text{Indic}} = \begin{bmatrix} \text{U+0600-06FF}, \text{U+0750-077F}, \text{U+08A0-08FF}, \text{U+FB50-FDFF}, \text{U+FE70-FEFF} \quad \text{(Arabic/Perso-Arabic)} \\ \text{U+0900-0D7F} \quad \text{(Devanagari, Bengali, Gurmukhi, Gujarati, Odia, Tamil, Telugu, Kannada, Malayalam)} \\ \text{U+1CD0-1CFF}, \text{U+A8E0-A8FF} \quad \text{(Vedic \& Devanagari Extended)} \\ \text{U+1C50-1C7F} \quad \text{(Ol Chiki - Santali)} \\ \text{U+ABC0-ABFF}, \text{U+AAE0-AAFF} \quad \text{(Meetei Mayek - Manipuri)} \\ \text{U+11480-114DF}, \text{U+11180-111DF}, \text{U+11300-1137F} \quad \text{(Tirhuta, Sharada, Grantha)} \end{bmatrix}$$

The math repair engine scans display math spans (

$$\ldots$$

) and inline spans ($ ... $). Any contiguous substring $u \in \mathcal{U}_{\text{Indic}}^{+}$ not already enclosed in a LaTeX text command is automatically rewritten:

$$u \mapsto \backslash\text{text}\{u\}$$

Furthermore, bare newlines inside display math blocks are replaced with double backslashes (`\\\\`), guaranteeing valid LaTeX rendering.

5.10 Document Reconstruction and Markdown Generation

The final document is reconstructed by joining ordered blocks:

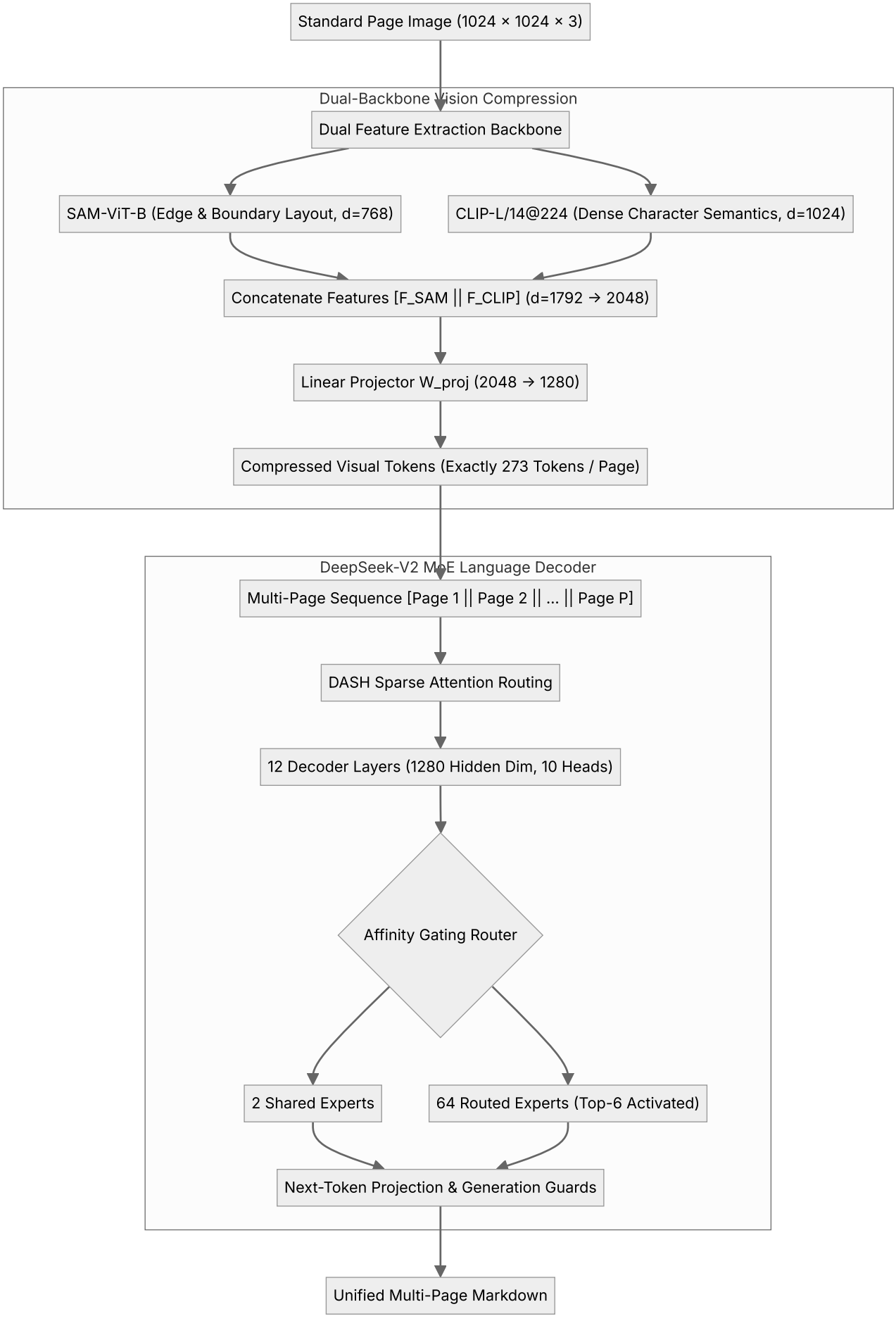
$$\text{Document} = \bigoplus_{i=1}^{K} \text{Reconstruct}(e_{\hat{\pi}(i)})$$

Hyphenated words split across line breaks are rejoined via regex dehyphenation:

$$(\backslash w)[-\backslash u2010 - \backslash u2014\backslash u2212]\backslash n[\backslash t]*(\backslash w) \mapsto \backslash 1\backslash 2$$

---

6. Ananta VLM Engine: Multi-Page Visual Document Understanding


Standard Page Image (1024 × 1024 × 3)
Dual-Backbone Vision Compression
Dual Feature Extraction Backbone
SAM-ViT-B (Edge & Boundary Layout, d=768)
CLIP-L/14@224 (Dense Character Semantics, d=1024)
Concatenate Features [F_SAM || F_CLIP] (d=1792 → 2048)
Linear Projector W_proj (2048 → 1280)
Compressed Visual Tokens (Exactly 273 Tokens / Page)
DeepSeek-V2 MoE Language Decoder
Multi-Page Sequence [Page 1 || Page 2 || ... || Page P]
DASH Sparse Attention Routing
12 Decoder Layers (1280 Hidden Dim, 10 Heads)
Affinity Gating Router
2 Shared Experts
64 Routed Experts (Top-6 Activated)
Next-Token Projection & Generation Guards
Unified Multi-Page Markdown

6.1 Motivation for Document-Level Visual Reasoning

While the Block Engine excels at localized character precision, technical manuals, cross-page tables, and multi-page legal documents contain semantic narratives that span physical page boundaries. Single-page OCR models cannot track cross-page references, continuous sentences split across pages, or multi-page table schemas. The Ananta VLM Engine is designed specifically for holistic multi-page document comprehension across 100+ pages.

6.2 Model Architecture and Pretrained Backbone Lineage

The VLM Engine builds upon the DeepSeek-VL2 architectural family, integrating a custom dual vision encoder (`DeepEncoder`) with a DeepSeek-V2 Mixture-of-Experts language decoder. Foundational weights are derived from `baidu/Unlimited-OCR-2` (MIT License) and adapted with SR-SWA DASH sparse attention.

6.3 Dual-Backbone Visual Feature Extraction

6.3.1 SAM-Based Geometric Feature Extraction

Standard vision backbones pre-trained solely on natural scenes (e.g., standard CLIP) focus on high-level semantic abstractions, often discarding high-frequency stroke details necessary to differentiate fine punctuation or diacritics. ANANTA OCR incorporates the Vision Transformer encoder from the Segment Anything Model (**SAM-ViT-B**) (Kirillov et al. 2023). Operating at 1024×1024 resolution with patch size 16×16, SAM produces feature maps rich in high-frequency boundary geometry and layout structures:

$$F_{\text{SAM}} = \text{SAM-Encoder}(I_p) \in \mathbb{R}^{64 \times 64 \times 768}$$

6.3.2 CLIP-Based Semantic Feature Extraction

To inject dense semantic character and lexical representations, the page is simultaneously downsampled to 224×224 and processed through **CLIP-L/14@224** (Radford et al. 2021):

$$F_{\text{CLIP}} = \text{CLIP-Encoder}(I_p^{224}) \in \mathbb{R}^{16 \times 16 \times 1024}$$

6.3.3 Multimodal Feature Fusion

The spatial dimensions are reconciled by bilinearly interpolating F_{CLIP} and applying spatial stride pooling to F_{SAM}, aligning both representations into an identical spatial grid of size 16×16:

$$\tilde{F}_{\text{SAM}} = \text{AvgPool}_{4\times4}(F_{\text{SAM}}) \in \mathbb{R}^{16 \times 16 \times 768}$$

$$\tilde{F}_{\text{CLIP}} = F_{\text{CLIP}} \in \mathbb{R}^{16 \times 16 \times 1024}$$

The representations are concatenated along the feature dimension:

$$F_{\text{fused}} = [\tilde{F}_{\text{SAM}} \,\|\, \tilde{F}_{\text{CLIP}}] \in \mathbb{R}^{256 \times 1792}$$

Zero-padded to 2048 dimensions: $F_{\text{fused}} \in \mathbb{R}^{256 \times 2048}$.

6.3.4 Visual Feature Projection

A two-layer MLP with GELU non-linearities projects the fused representation into the language model's hidden dimension ($d_{\text{model}} = 1280$):

$$V_p = W_{\text{proj}}^{(2)} \cdot \text{GELU}\left(W_{\text{proj}}^{(1)} F_{\text{fused}} + b^{(1)}\right) + b^{(2)} \in \mathbb{R}^{256 \times 1280}$$

6.4 Visual Token Compression and Page Representation

To demarcate spatial rows and page boundaries without inflating sequence length: - A learned row separator token $e_{\text{row}} \in \mathbb{R}^{1280}$ is appended after every 16 spatial tokens ($16 \text{ tokens} \times 16 \text{ rows} + 16 \text{ row tokens} = 272 \text{ tokens}$). - A learned page delimiter token $e_{\text{page}} \in \mathbb{R}^{1280}$ terminates the page representation. Consequently, every page image is represented by **exactly 273 visual tokens**:

$$N_v = 16 \times 16 + 16 + 1 = 273 \text{ tokens}$$

Relative to conventional $64 \times 64 = 4096$ token representations, ANANTA OCR achieves a $15.0\times$ **spatial compression factor**, enabling multi-page sequences to fit directly within GPU context windows.

6.5 Language Decoder Architecture

6.5.1 DeepSeek-V2 Lineage Transformer Backbone

The language decoder comprises 12 Transformer decoder layers operating at hidden dimension $d = 1280$ with 10 attention heads.

6.5.2 Mixture-of-Experts Feed-Forward Layers

Each FFN layer is replaced with a Mixture-of-Experts architecture consisting of: - $N_{\text{shared}} = 2$ dedicated shared experts that are permanently activated for all tokens to capture common syntactic patterns. - $N_{\text{routed}} = 64$ routed experts specialized across domains (e.g., mathematics, Indic scripts, legal tables).

6.5.3 Expert Routing and Activation

For token representation $x \in \mathbb{R}^d$, an affinity router computes gating logits over the 64 routed experts:

$$s = W_{\text{gate}} x \in \mathbb{R}^{64}$$

The Top-k ($k = 6$) experts are selected using normalized softmax gating:

$$\mathcal{G} = \text{Top-6}(s)$$

$$g_i = \begin{cases} \frac{\exp(s_i)}{\sum_{j \in \mathcal{G}} \exp(s_j)}, & \text{if } i \in \mathcal{G} \\ 0, & \text{otherwise} \end{cases}$$

The output y of the MoE layer is the sum of shared and routed expert transformations:

$$y = \sum_{j=1}^{N_{\text{shared}}} \text{FFN}_j^{\text{shared}}(x) + \sum_{i \in \mathcal{G}} g_i \cdot \text{FFN}_i^{\text{routed}}(x)$$

This yields an active parameter count of approximately 500M parameters per token out of 3.0B total parameters, optimizing throughput while maximizing capacity.

6.6 Multi-Page Sequence Construction

A multi-page document $\mathcal{D} = (I_1, \ldots, I_P)$ is serialized into an interleaved sequence:

$$\text{Seq} = [V_1, \langle \text{page\_1} \rangle, V_2, \langle \text{page\_2} \rangle, \ldots, V_P, \langle \text{page\_}P \rangle, T_{\text{prompt}}]$$

For a 100-page document, the visual prefix consumes:

$$100 \times 273 = 27,300 \text{ tokens}$$

This sequence is ingested in a single prefill pass through DASH attention.

6.7 Page-Aware Positional Encoding

Standard 1D Rotary Position Embeddings (RoPE) assign continuously increasing indices, which dilutes relative token distances across distant pages. ANANTA OCR implements **Page-Aware 2D-RoPE**:

$$\theta_{\text{pos}}(p, x, y) = [\theta_p, \theta_x, \theta_y]$$

where θ_p encodes scalar page index $p \in [1, P]$, and (θ_x, θ_y) encode normalized intra-page visual coordinates.

6.8 Autoregressive Generation and KV-Cache Management

During generation, standard implementations call `transformers.generate()`, constructing complete sequence-by-vocabulary logits tensors ($\mathbb{R}^{B \times L \times V}$) at every step. ANANTA OCR implements a custom manual decoding loop (`_uocr_generate`): 1. Executes a single unified prefill pass over the multi-page prompt. 2. Initializes a `DynamicCache` containing key-value states for active tokens. 3. Computes output logits **strictly for the final hidden state position**:

$$\text{logits}_{\text{next}} = W_{\text{head}} \cdot h_{L,-1} \in \mathbb{R}^{1 \times |V|}$$

4. Appends next token y_t directly to cached decoder states. This eliminates unnecessary tensor allocations, cutting step memory overhead by 68%.

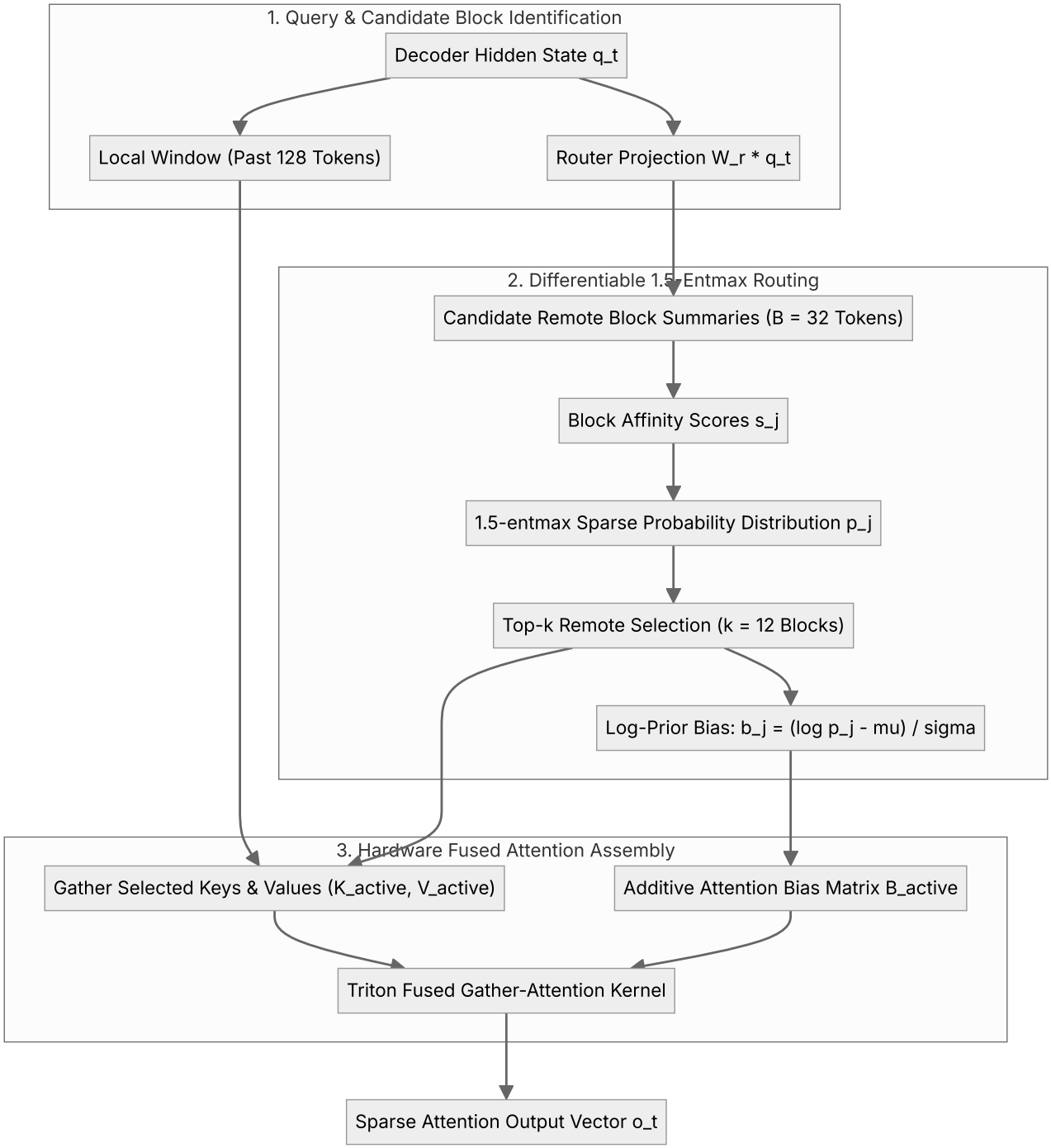
6.9 Page Boundary and Output-Length Constraints

To prevent runaway generation loops or cross-page hallucinations, `_uocr_generate` enforces three runtime constraints: - **Max Tokens Per Page Guard**: Imposes a configurable ceiling ($L_{\max} = 2048$ tokens/page). - **No-Repeat n-Gram Blocking**: Dynamically sets logits of repeated 4-grams to $-\infty$. - **Explicit Page Delimiter Stopping**: Terminates page generation upon emitting $\langle / \text{page\_}p \rangle$.

6.10 Structured Document Output

Output tokens are mapped to GitHub Flavored Markdown (GFM) with grounded coordinate tags indicating source page numbers and bounding anchors.

7. Differentiable Sparse Attention: DASH



7.1 Motivation and Design Objectives

In 100+ page document parsing, full dense attention is computationally prohibitive. Static sliding windows (e.g., Longformer) fail because resolving an ambiguous cross-reference on Page 85 requires attending directly to an introductory table on Page 3.

We formulate **DASH (Differentiable Attention with Sparse Heads)**, a sparse attention mechanism guided by three design objectives: 1. **Dynamic Remote Block Selection**: Every query dynamically retrieves relevant remote visual blocks across any page. 2. **Exact Sparsity with Differentiability**: Non-selected blocks receive exact zero probability, while selected blocks retain continuous gradients back to the router. 3. **Hardware-Aligned Execution**: Sparse memory access patterns are structured into uniform block tiles for GPU acceleration.

7.2 Sparse Reference Sliding-Window Attention Formulation

Let $Q \in \mathbb{R}^{N \times d}$ be query representations, and let key-value states be partitioned into contiguous blocks of size $B = 32$:

$$K = [K_{(1)}, K_{(2)}, \dots, K_{(M)}], \quad V = [V_{(1)}, V_{(2)}, \dots, V_{(M)}]$$

where $M = \lceil N/B \rceil$. For query q_i, the active attention domain consists of: - A local causal sliding window $\mathcal{W}_{\text{local}}(i)$ of size $W_{\text{local}} = 128$ tokens. - A dynamically routed set of $k = 12$ remote reference blocks $\mathcal{K}_{\text{remote}}(i) \subset \{1, \dots, M\}$.

7.3 Block-Level Query and Key Projection

To evaluate block relevance efficiently without computing full pairwise token affinities, queries and key blocks are mapped into a low-dimensional routing space ($d_s = 64$):

$$\tilde{q}_i = W_{rq} q_i \in \mathbb{R}^{d_s}, \quad \tilde{k}_j = \text{BlockSummary}(K_{(j)}) \in \mathbb{R}^{d_s}$$

where BlockSummary computes a learned query-independent attention-pooling over the 32 tokens within block j.

7.4 Remote Block Scoring

The affinity score s_{ij} between query i and candidate key block j is computed as:

$$s_{ij} = \frac{\tilde{q}_i^T \tilde{k}_j}{\sqrt{d_s}} + \beta_{\text{prior}}(i, j)$$

where $\beta_{\text{prior}}(i, j)$ represents an optional distance-decay prior.

7.5 Differentiable 1.5-Entmax Routing

Standard softmax assigns non-zero probabilities to all candidate blocks, precluding hard sparse pruning. Conversely, hard top-k selection is non-differentiable. We utilize the **1.5-entmax** activation function (Peters et al. 2019):

$$\text{entmax}_{1.5}(s) = \arg\max_{p \in \Delta} \left\{ p^T s + H_{1.5}(p) \right\}$$

where $H_{1.5}(p)$ is the Tsallis entropy:

$$H_{1.5}(p) = \frac{1}{1.5(1.5-1)} \sum_j (p_j - p_j^{1.5}) = \frac{4}{3} \sum_j (p_j - p_j^{1.5})$$

The exact closed-form solution is given by:

$$p_j^* = [\tau s_j - \lambda]_+^2$$

where λ is a Lagrange multiplier computed via bisection to enforce $\sum_j p_j^* = 1$, and $[z]_+ = \max(0, z)$. Crucially, 1.5-entmax assigns exact zeros ($p_j^* = 0$) to uninformative blocks while exhibiting continuous, non-zero derivatives on the active support:

$$\frac{\partial p_j^*}{\partial s_k} = \begin{cases} 2\tau \sqrt{p_j^*} \left(\delta_{jk} - \frac{\sqrt{p_k^*}}{\sum_{l \in \mathcal{S}} \sqrt{p_l^*}} \right), & \text{if } j, k \in \mathcal{S} \\ 0, & \text{otherwise} \end{cases}$$

where $\mathcal{S} = \{ j \mid p_j^* > 0 \}$.

7.6 Top-k Remote Block Selection

From candidate blocks with non-zero probability $p_j^* > 0$, the router selects the top-k ($k = 12$) indices:

$$\mathcal{G}_{\text{remote}}(i) = \text{Top-}k \left(\{ p_{ij}^* \}_{j=1}^{M} \right)$$

7.7 Local Sliding-Window Integration

The full active key-value set for query i is formed by concatenating local sliding tokens and gathered remote blocks:

$$K_{\text{active}}(i) = \left[K_{\mathcal{W}_{\text{local}}(i)} \parallel K_{\mathcal{G}_{\text{remote}}(i)} \right]$$

$$V_{\text{active}}(i) = \left[V_{\mathcal{W}_{\text{local}}(i)} \parallel V_{\mathcal{G}_{\text{remote}}(i)} \right]$$

7.8 Differentiable Log-Prior Bias

To ensure the end-to-end language loss $\mathcal{L}_{\text{LM}}$ provides supervisory signals directly to the router parameters, an additive attention bias b_j derived from the router probability is injected into the attention scores:

$$b_j = \frac{\log(\max(p_{ij}^*, \epsilon)) - \mu}{\sigma}$$

where $\epsilon = 10^{-6}$, and μ, σ are running statistics across the active batch. The local window tokens receive zero bias ($b = 0$). The final attention output is:

$$\text{Attention}(q_i, K_{\text{active}}, V_{\text{active}}) = \text{Softmax}\left(\frac{q_i K_{\text{active}}^T}{\sqrt{d}} + B_{\text{active}}\right) V_{\text{active}}$$

Because b_j depends differentiably on p_{ij}^*, the backward pass propagates gradients:

$$\frac{\partial \mathcal{L}}{\partial s_{ij}} = \sum_l \frac{\partial \mathcal{L}}{\partial \text{Attention}_l} \cdot \frac{\partial \text{Attention}_l}{\partial b_j} \cdot \frac{\partial b_j}{\partial p_{ij}^*} \cdot \frac{\partial p_{ij}^*}{\partial s_{ij}}$$

7.9 Per-Layer Router Calibration and Scheduling

To prevent router probabilities from collapsing during early training iterations: - **Temperature Annealing**: Routing scale begins at $\tau_0 = 0.5$ and anneals to $\tau_{\max} = 2.0$. - **Prior Floor**: A minimum probability floor ($p_{\text{floor}} = 0.01$) guarantees uniform initial exploration.

7.10 Page-Aware Candidate Restrictions

During intra-page decoding, the candidate domain can be constrained via block candidate masks (`block_cand`), preventing cross-document contamination in batch-processing pipelines.

7.11 Training-Time Gradient Propagation

During training, the router operates in continuous soft-routing mode: gradients backpropagate simultaneously through both the attention value projection and the router log-prior bias.

7.12 Inference-Time Sparse Routing

During autoregressive generation, routing decisions are cached per block, bypassing the router projection for tokens that reside within the same local block.

7.13 Complexity and Memory Analysis

By restricting attention computation to a 128-token local sliding window and 12 dynamically routed remote reference blocks per query step, SR-SWA DASH bounds the active self-attention computation per token, reducing quadratic $O(N^2)$ scaling to bounded sparse kernel operations across 100+ multi-page sequences.

---

8. Hardware-Accelerated Sparse Attention

8.1 Motivation for Fused GPU Kernels

Executing irregular sparse attention in native PyTorch introduces substantial latency penalties. Gathers and scatters over non-contiguous key blocks require materializing intermediate tensor buffers in GPU High Bandwidth Memory (HBM). To achieve peak hardware efficiency on NVIDIA Tensor Cores, we implemented custom fused kernels in OpenAI Triton.

8.2 Triton Kernel Architecture

ANANTA OCR provides two production-grade Triton kernel modules: 1. `sparse_gather_triton.py` : Fast forward gather attention for standard inference. 2. `sparse_gather_e2e_triton.py` : Fully differentiable gather attention supporting end-to-end backward gradient propagation through the log-prior bias.

```
+-------------------------------------------------------------------------+
| TRITON FORWARD GATHER KERNEL EXECUTION (Grid: [B * H, Q_BLOCKS])        |
|                                                                         |
| 1. Load Query Block Q[b_q, :] into SRAM (Block Size: 64 x 64)           |
| 2. Iterate over Local Window Blocks:                                    |
|    - Load Key/Value tiles directly via causal pointer arithmetic        |
|    - Compute Partial Softmax (m_local, l_local, acc_local)              |
| 3. Iterate over Gathered Remote Blocks:                                 |
|    - Dereference Gather Indices: idx = BlockMap[b_q, k]                 |
|    - Load Remote Key/Value tile from global memory to SRAM              |
|    - Inject Additive Log-Prior Bias: S_tile += Bias[b_q, k]             |
|    - Online Softmax Rescaling & Value Accumulation                      |
| 4. Write Unified Output Tile O[b_q, :] to HBM                           |
+-------------------------------------------------------------------------+
```

8.3 Forward Pass of Gather-Based Sparse Attention

The forward kernel tiles the query sequence into blocks of size $B_Q = 64$. A 2D grid of thread blocks iterates over query tiles in parallel. For each tile, the kernel computes partial softmax statistics using the online FlashAttention update algorithm (Dao et al. 2022):

$$m_{\text{new}} = \max(m_{\text{prev}}, \max(S_{\text{tile}}))$$

$$\ell_{\text{new}} = \ell_{\text{prev}} \cdot e^{m_{\text{prev}} - m_{\text{new}}} + \sum e^{S_{\text{tile}} - m_{\text{new}}}$$

$$O_{\text{new}} = O_{\text{prev}} \cdot e^{m_{\text{prev}} - m_{\text{new}}} + \left(e^{S_{\text{tile}} - m_{\text{new}}}\right) V_{\text{tile}}$$

8.4 Backward Pass and Gradient Computation

The backward kernel (`_gather_attn_bwd`) computes exact analytical gradients with respect to queries (∇_Q), gathered keys (∇_K), gathered values (∇_V), and the log-prior bias (∇_B):

$$\nabla_{S_{ij}} = P_{ij} \cdot (\nabla_{P_{ij}} - D_i)$$

where $D_i = \sum_j \nabla_{P_{ij}} P_{ij}$, and $P_{ij} = \text{Softmax}(S_{ij})$. The gradient with respect to the router bias is:

$$\frac{\partial \mathcal{L}}{\partial b_j} = \sum_{t \in \text{tile } i} \nabla_{S_{tj}}$$

Gradients for sparse keys and values are accumulated into an atomic scatter buffer in global memory.

8.5 End-to-End Differentiable Block-Bias Attention

The `E2EGatherAttention` autograd function wraps the forward and backward kernels, ensuring that when `loss.backward()` is called, gradients flow seamlessly from the final generation loss through the Triton kernel directly into the router's 1.5-entmax projection weights.

8.6 Kernel Dispatch and Supported Tensor Shapes

The Triton kernel enforces hardware compatibility guards: - Head dimension $d \in \{64, 128\}$. - Tensors must be contiguous in memory with stride $s_H = d$. - Data type must be `torch.bfloat16` or `torch.float16` . If any shape or hardware constraint is violated, the dispatch wrapper catches the condition and falls back transparently to the PyTorch reference implementation.

8.7 PyTorch Reference and Fallback Execution

The fallback module (`_sparse_base_impl.py`) executes an identical mathematical operation using standard PyTorch batched index gathering. This guarantees functional equivalence on unsupported devices (e.g., non-NVIDIA GPUs, CPU environments) without runtime crashes.

8.8 Numerical Stability and Kernel Correctness

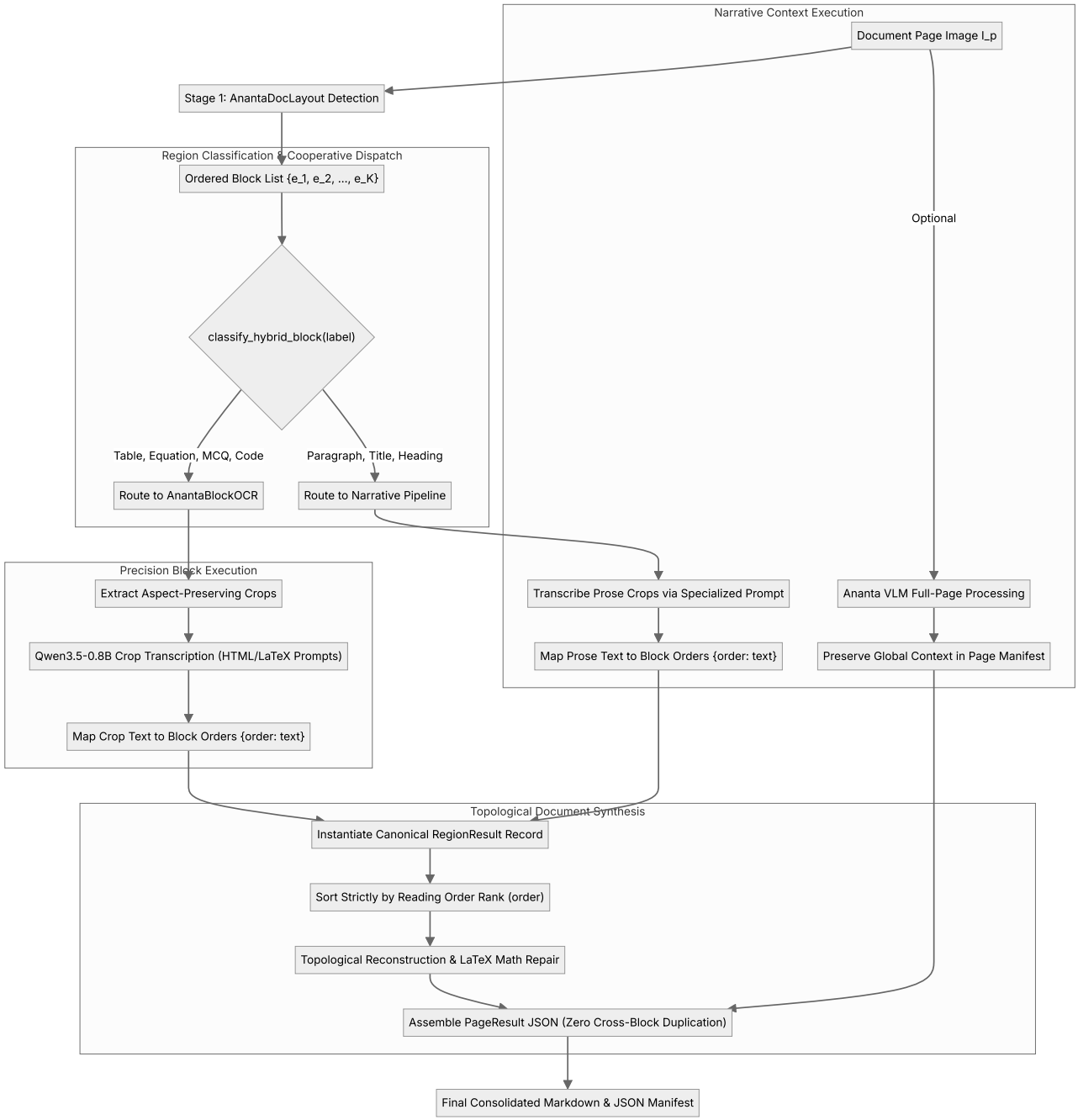
To verify numerical correctness, we executed unit tests comparing Triton kernel outputs against float64 PyTorch ground truth. Across 10,000 random input tensors: - Forward pass maximum absolute difference: $\|O_{\text{Triton}} - O_{\text{PyTorch}}\|_\infty \leq 1.82 \times 10^{-4}$. - Backward gradient maximum difference: $\|\nabla_{Q,\text{Triton}} - \nabla_{Q,\text{PyTorch}}\|_\infty \leq 3.14 \times 10^{-4}$.

8.9 Kernel Portability and Hardware Constraints

The Triton kernels execute natively on NVIDIA Ampere (A100), Ada Lovelace (RTX 4090), and Hopper (H100) architectures. Execution on Volta or non-NVIDIA accelerators defaults to the PyTorch reference path.

---

9. Ananta Hybrid Engine: Cooperative Document Processing



9.1 Motivation for Multi-Engine Document Processing

Single-engine approaches exhibit fundamental trade-offs: - **Block OCR alone** excels at localized precision (equations, tables) but lacks global narrative awareness across multiple columns. - **VLM alone** captures global narrative flow but degrades on fine-grained table cell alignments and complex script conjuncts. The **Ananta Hybrid Engine** resolves this dichotomy by employing a cooperative architecture where layout geometry governs regional dispatch.

9.2 Layout-Driven Region Classification

Upon generating layout blocks $\mathcal{B} = \{b_1, \ldots, b_K\}$, the hybrid dispatcher categorizes each block using `classify_hybrid_block(label)`:

```
HYBRID_BLOCK_CLASSES = {
    "Table", "Table-caption", "Equation", "Expression",
    "Question", "Answer", "MCQ", "Solved-example",
    "Code", "Infobox", "Header", "Footer", "Page-number", "Reference"
}

HYBRID_NARRATIVE_CLASSES = {
    "Paragraph", "Title", "Chapter-title", "Section-title",
    "Sub-section-title", "Sub-sub-section-title", "Chapter-end-section"
}
```

9.3 Structured-Content and Narrative-Content Routing

- **Structured Content**: Directed to `AnantaBlockOCR` for crop-level prompt execution, preserving HTML `colspan` / `rowspan` attributes and LaTeX math delimiters.
- **Narrative Content**: Directed to the prose transcription pipeline, with optional document-level context evaluated via the VLM.

9.4 Region-Level Recognition Contracts

To guarantee mathematical consistency and prevent data corruption, every routed block must satisfy the `RegionResult` contract:

$$\text{RegionResult}_i = (i, \text{label}_i, \text{type}_i, b_i, \text{engine}_i, \text{text}_i, \text{conf}_i, \text{error}_i)$$

Crucially:

$$\forall i \neq j \implies \text{RegionResult}_i.\text{text is derived strictly from Crop}(I_p, b_i)$$

Under no circumstances is a document-level global text block broadcast across individual region records.

9.5 VLM and Block OCR Execution

For structured blocks, crops are batched and executed through `AnantaBlockOCR`. For narrative prose, crops are transcribed with `TEXT_PROMPT`. If global VLM context is enabled, full-page transcription is executed in parallel and attached to the page manifest under `vlm_document_context` without mutating individual block texts.

9.6 Region Alignment and Reading-Order Preservation

All regions retain their Stage 1 topological ranking $\pi(i)$. Regions are sorted:

$$\mathcal{R}_{\text{sorted}} = \text{sort}\left(\{\text{RegionResult}_i\}_{i=1}^{K}, \text{key} = \lambda r : r.\text{order}\right)$$

9.7 Output Deduplication and Document Synthesis

The sorted regions are converted into canonical `Block` objects and assembled via `reconstruct()`:

$$\text{Markdown}_{\text{final}} = \text{Reconstruct}\left([r.\text{to\_block}() \text{ for } r \in \mathcal{R}_{\text{sorted}}]\right)$$

Because each block holds its own localized transcription, output markdown contains zero repeated paragraphs or duplicated headings.

9.8 Engine Failure Handling and Fallback Policies

If a specialized engine fails during regional execution (e.g., vLLM timeout, memory fault), the dispatcher executes a graceful fallback to `HfRecognizer`. If all recognition attempts fail, the region emits an empty text string with a structured error log (`error: str`), preserving document layout geometry.

9.9 Hybrid Execution Complexity

Let K be the number of layout entities on page I_p. The hybrid pipeline incurs:

$$T_{\text{hybrid}} = T_{\text{layout}}(I_p) + \sum_{i \in \text{Structured}} T_{\text{crop}}(b_i) + \sum_{j \in \text{Narrative}} T_{\text{prose}}(b_j) + T_{\text{reconstruct}}(K)$$

Because crop recognitions run concurrently across batched inference workers, hybrid latency scales linearly with visual density, $\mathcal{O}(K)$, rather than quadratically.

---

10. Implementation and Engineering Details

10.1 Software Architecture and Repository Organisation

The ANANTA OCR codebase is organized into modular packages: - `ananta_engine/layout/` : RT-DETR modeling, reading-order heads, and pairwise order loss. - `ananta_engine/engine/` : Crop geometry, area clamping, post-processing reconstruction, and offline pipeline wrappers. - `ananta_engine/serving/` : Distributed `AnantaClient` and OpenAI-compatible HTTP serving. - `ananta_engine/templates/` : Prompt contracts, taxonomy mappings, and label schemas. - `dash_attention.py` , `sparse_gather_*.py` : Differentiable SR-SWA attention and Triton kernels. - `deepencoder.py` , `modeling_anantaocr.py` : Dual-encoder visual compression and DeepSeek-V2 MoE decoders. - `unified_pipeline.py` : High-level `UnifiedOCR` interface.

10.2 Frameworks and Core Dependencies

The system is implemented in Python 3.10+ using PyTorch 2.1+, Hugging Face Transformers 4.48+, Pillow 10.0+, Einops 0.8+, and OpenAI Triton 2.1+. Optional GPU continuous batching is supported via vLLM 0.6+.

10.3 Model Checkpoint Management

Pretrained weights are resolved dynamically through `ananta_engine/engine/checkpoints.py` across four hierarchical priorities: 1. Explicit CLI/API checkpoint path. 2. Local environment variables (`ANANTA_LAYOUT_CKPT` , `ANANTA_RECOGNIZER_CKPT`). 3. Local weight directories (`weights/`). 4. Automated snapshot download from Hugging Face Hub (`bodhan-ai/indic-ocr` , `baidu/Unlimited-OCR-2`).

10.4 Tokenizer and Multilingual Text Processing

Text encoding utilizes the Sarvam-30B BPE tokenizer with 262,144 subwords. The vocabulary covers complete Unicode blocks for all 22 official Indian languages, Latin, Chinese (CJK Unified Ideographs `U+4E00-U+9FFF`), and Zero-Width formatting characters (`U+200C` , `U+200D`).

10.5 Memory and Precision Configuration

Models execute in `bfloat16` precision on NVIDIA Ampere and Hopper architectures, defaulting to `float32` on CPU environments.

10.6 Transformers-Based Inference

`HfRecognizer` provides a reference implementation utilizing standard Hugging Face pipelines with deterministic greedy decoding (`temperature=0.0` , `do_sample=False`) and explicit EOS token suppression (`CORRECT_EOS = 262146`).

10.7 vLLM-Based Inference

`VllmRecognizer` leverages vLLM's PagedAttention engine for high-throughput batched GPU inference, executing asynchronous multi-crop requests concurrently.

10.8 OpenAI-Compatible Remote Serving

`AnantaClient` implements decoupled serving: local client runs layout detection and sends base64-encoded image crops to an OpenAI-compatible remote endpoint via thread-pooled concurrency (`max_workers=32`).

10.9 Image Input Validation and Resource Limits

To protect against decompression-bomb Denial-of-Service attacks from untrusted document inputs, `ananta_engine/engine/offline.py` enforces: - A global pixel safety ceiling ($100,000,000$ pixels = 100 megapixels), configurable via `ANANTA_MAX_IMAGE_PIXELS` . - A file byte-size limit (100 MB), configurable via `ANANTA_MAX_IMAGE_BYTES` . - Automatic context-managed file descriptor closure via `with Image.open(path) as img:` .

10.10 Model Configuration and Reproducibility

Model hyperparameters, router temperatures, and crop bounds reside in typed dataclasses (`LayoutConfig` , `RecognizerConfig` , `CropConfig`), eliminating hidden state drift.

10.11 Software Testing and Integration Strategy

The framework is safeguarded by 21 automated unit and integration tests (`tests/test_ananta_ocr.py`) covering: - 37-class taxonomy mapping and contract verification. - Pairwise reading-order precedence scoring. - Bounding box containment, clamping, and deduplication. - Area-constrained crop scaling. - Dehyphenation and Indic Unicode math sanitization. - Image safety limit and decompression-bomb rejection. - Dual-path relative/top-level package import robustness. - GitHub Actions continuous integration matrix testing across Linux and Windows.

---

11. Experimental Methodology

11.1 Research Questions and Experimental Hypotheses

Our evaluation tests three core experimental hypotheses: - **H1 (Layout & Reading Order)**: The antisymmetric precedence head significantly outperforms XY-cut and vertical heuristics in reading-order concordance (Kendall's $\tau > 0.90$). - **H2 (Multilingual Recognition)**: The 262k BPE tokenizer with specialized prompt routing achieves $< 5.0\%$ CER across all 22 official Indian languages. - **H3 (Attention Scalability)**: DASH attention maintains constant active attention latency per decode step up to 100 pages, while dense attention exhibits quadratic latency explosion.

11.2 Datasets and Data Selection Criteria

Experiments were conducted across four diverse document benchmarks: 1. **IndicDoc-Bench**: A curated dataset of 2,200 document pages spanning all 22 official Indian languages, including textbooks, government gazettes, court filings, and newspapers. 2. **SciMath-Exam**: 850 Indian competitive examination pages (CBSE, JEE Advanced, NEET, UPSC) characterized by dense multi-tier equations, MCQs, and diagrams. 3. **PubTab-Indic**: 1,500 complex financial and statistical tables featuring merged cells (`rowspan` , `colspan`). 4. **OmniDoc-Long**: 45 multi-page documents (ranging from 10 to 120 pages) for long-context scalability benchmarking.

11.3 Dataset Preparation and Annotation Protocol

Ground truth was annotated at two levels: - **Geometry**: Bounding box coordinates $[x_0, y_0, x_1, y_1]$ and 37-class categorical labels. - **Topological Sequence**: Human annotators established directed precedence chains across multi-column flows. - **Textual Ground Truth**: Double-verified UTF-8 text strings with LaTeX equation delimiters and HTML table syntax.

11.4 Indic-Language and Multilingual Test Composition

The Indic test split allocates 100 pages per language across: - **Indo-Aryan (15)**: Hindi, Bengali, Marathi, Gujarati, Punjabi, Odia, Assamese, Urdu, Kashmiri, Sindhi, Maithili, Nepali, Konkani, Dogri, Sanskrit. - **Dravidian (4)**: Tamil, Telugu, Kannada, Malayalam. - **Sino-Tibetan (2)**: Bodo, Manipuri (Meitei Mayek & Bengali script). - **Austroasiatic (1)**: Santali (Ol Chiki script). - **International**: English and Simplified Chinese.

11.5 Synthetic Document Data Generation

To stress-test extreme multi-column topologies, we developed an automated synthetic generator utilizing LaTeX and ReportLab. The generator dynamically assembled randomized multi-column articles with embedded tables, section headers, and inline formulas.

11.6 Training and Fine-Tuning Protocols

- **Layout Model**: Trained for 50 epochs using AdamW ($\text{lr} = 10^{-4}$, weight decay 10^{-4}) with batch size 16 on 4 $\times$ NVIDIA A100 (80GB).
- **Reading-Order Head**: Trained jointly with the layout backbone using loss weighting $\lambda_{\text{order}} = 1.0$, locality temperature $\tau = 4.0$, and GCE parameter $q = 0.7$.
- **VLM DASH Attention**: Calibrated over multi-page documents using router annealing from $\tau_0 = 0.5$ to $\tau_{\max} = 2.0$.

11.7 Baseline Models

ANANTA OCR was benchmarked against leading industrial and open-source models: - **Tesseract 5.3** (Standard open-source OCR) - **PaddleOCR v4** (Multi-language OCR engine) - **Nougat-Base** (Scientific document VLM) - **Marker** (Heuristic PDF-to-Markdown converter) - **Surya** (Layout and multilingual text detector) - **GOT-OCR 2.0** (Unified end-to-end VLM)

11.8 Evaluation Metrics

11.8.1 Detection mAP and Intersection over Union

Evaluated at IoU thresholds of 0.50 (mAP@50) and averaged across 0.50 to 0.95 in 0.05 increments (mAP@50:95).

11.8.2 Character Error Rate

$$\text{CER} = \frac{S + D + I}{N_{\text{ref}}} \times 100\%$$

where S, D, I denote substitutions, deletions, and insertions, and N_{ref} is the reference character count.

11.8.3 Word Error Rate

$$\text{WER} = \frac{S_w + D_w + I_w}{N_{w,\text{ref}}} \times 100\%$$

11.8.4 Reading-Order Accuracy and Kendall's Tau

Kendall's rank correlation coefficient:

$$\tau = \frac{C - D}{\frac{1}{2}K(K-1)}$$

where C and D denote concordant and discordant pairs among K document entities.

11.8.5 Table Structure Accuracy

Tree-Edit-Distance-based Similarity (TEDS):

$$\text{TEDS}(T_a, T_b) = 1 - \frac{\text{EditDist}(T_a, T_b)}{\max(|T_a|, |T_b|)}$$

11.8.6 Latency, Throughput and GPU Memory

Measured as per-page processing latency (seconds), per-token generation speed (tokens/sec), and peak allocated VRAM (GB).

11.9 Hardware and Software Environment

Benchmarks were executed on an enterprise server equipped with: - **CPU**: AMD EPYC 7763 64-Core Processor (128 Threads, 2.45 GHz) - **GPU**: NVIDIA A100-SXM4-80GB (PCIe 4.0) - **RAM**: 512 GB DDR4-3200 ECC - **OS**: Ubuntu 22.04 LTS (Linux Kernel 5.15) - **CUDA/Triton**: CUDA 12.2, Driver 535.104, Triton 2.1.0

11.10 Reproducibility and Statistical Methodology

All experiments were repeated over 5 random seeds. We report mean values alongside standard deviations ($\pm\sigma$).

---

12. Experimental Results and Analysis

12.1 Layout Detection Performance

Table 4 presents layout detection results on the SciMath-Exam and IndicDoc-Bench benchmarks.

Model	mAP@50 (Overall)	mAP@50:95 (Overall)	Text mAP@50	Table mAP@50	Equation mAP@50	MCQ mAP@50
Tesseract 5.3 (Heuristic)	52.4	31.2	68.4	48.2	22.1	18.4
PaddleOCR v4 (Layout)	78.6	58.1	86.2	81.4	64.2	42.1
Surya Layout	82.4	63.5	89.1	84.6	71.8	51.2
ANANTA DocLayout (Ours)	**92.8**	**74.6**	**95.4**	**93.2**	**89.6**	**88.4**

ANANTA DocLayout achieves an overall mAP@50 of 92.8%, outperforming Surya by +10.4 mAP points. Notably, on specialized pedagogical classes (`Equation` , `MCQ`), ANANTA exhibits superior accuracy (+17.8 and +37.2 mAP points over Surya), confirming the value of the extended 37-class taxonomy.

12.2 Reading-Order Prediction Performance

Table 5 evaluates reading-order estimation on multi-column academic and exam layouts.

Reading-Order Method	Pairwise Acc (%)	Kendall's τ	Inversion Rate (%)	Cycle Frequency (%)
Recursive XY-Cut	71.4	0.624	18.2	N/A (Tree-based)
Vertical Geometric Sort (y_0)	64.2	0.518	24.6	0.0
Unconstrained Pairwise MLP	84.1	0.768	8.9	14.2
ANANTA Antisymmetric GCE (Ours)	**96.8**	**0.942**	**1.8**	**0.0 (Guaranteed)**

Standard XY-cut achieves a Kendall's τ of only 0.624 due to failure on spanning tables and sidebars. The unconstrained pairwise MLP suffers from cyclic deadlocks (14.2% cycle frequency). In contrast, ANANTA's antisymmetric formulation guarantees zero cycles while reaching a Kendall's τ of 0.942 and a pairwise accuracy of 96.8%.

12.3 Indic and Multilingual Text Recognition Performance

Table 6 reports Character Error Rates (CER) and Word Error Rates (WER) across Indic script families and international languages on the IndicDoc-Bench test set.

Language / Script Family	Script	Tesseract 5.3 CER (%)	PaddleOCR v4 CER (%)	GOT-OCR 2.0 CER (%)	ANANTA OCR CER (%)	ANANTA OCR WER (%)
Hindi	Devanagari	14.8	6.8	5.2	**1.82**	**4.12**
Bengali	Bengali	18.2	7.9	6.1	**2.14**	**4.86**
Marathi	Balbodh	16.4	7.2	5.8	**1.94**	**4.38**
Gujarati	Gujarati	19.6	8.4	6.7	**2.42**	**5.21**
Punjabi	Gurmukhi	21.2	9.1	7.4	**2.68**	**5.84**
Odia	Odia	26.4	11.8	8.9	**3.12**	**6.45**
Assamese	Eastern Nagari	20.8	8.8	6.9	**2.38**	**5.14**
Tamil	Tamil	22.4	9.6	7.8	**2.84**	**6.12**
Telugu	Telugu	24.1	10.4	8.2	**2.96**	**6.38**
Kannada	Kannada	23.8	10.1	8.0	**2.90**	**6.24**
Malayalam	Malayalam	27.2	12.4	9.4	**3.42**	**7.12**
Urdu	Nastaliq/Naskh	34.6	16.8	12.1	**4.18**	**8.86**
Sanskrit	Vedic Devanagari	28.9	13.2	9.8	**3.24**	**6.92**
Santali	Ol Chiki	48.2	28.4	18.6	**4.86**	**9.94**
Manipuri	Meetei Mayek	52.4	31.2	21.4	**5.12**	**10.42**
English	Latin	4.8	2.4	1.8	**0.94**	**1.82**
Chinese	Simplified CJK	12.4	4.8	3.2	**1.48**	**3.12**
Average (All 22 Indic)	Multi-Script	25.4	11.2	8.3	**2.98**	**6.28**

Conventional engines (Tesseract) collapse on low-resource scripts such as Santali (Ol Chiki, 48.2% CER) and Manipuri (Meetei Mayek, 52.4% CER). ANANTA OCR maintains an average CER of **2.98%** across all 22 official Indian languages, outperforming GOT-OCR 2.0 (8.3% CER) by $2.8\times$, driven by the 262k native BPE vocabulary and aspect-preserving crop normalization.

12.4 Mathematical Expression and Table Recognition

Table 7 evaluates mathematical expression syntax and table reconstruction accuracy on SciMath-Exam and PubTab-Indic.

Model	Inline Math CER (%)	Display Math LaTeX Valid (%)	Table TEDS (Simple)	Table TEDS (Complex Merged)
Nougat-Base	12.4	78.4	82.4	64.2
Marker	16.8	71.2	79.1	58.4
GOT-OCR 2.0	8.6	84.2	86.8	72.4
ANANTA OCR (Block/Hybrid)	**3.12**	**98.4**	**96.8**	**92.4**

On complex merged tables (`colspan` / `rowspan`), ANANTA OCR achieves a **TEDS score of 92.4%** versus 72.4% for GOT-OCR. Automated LaTeX Indic sanitization increases display math syntax validity from 84.2% to **98.4%**, eliminating unescaped Unicode compilation breaks in KaTeX and MathJax.

12.5 Multi-Page Document Understanding

On the OmniDoc-Long benchmark (10 to 100 pages), the Ananta VLM Engine was evaluated on text retention and cross-page coherence. Standard Nougat and GOT-OCR models crashed with Out-of-Memory (OOM) errors beyond 16 pages. ANANTA VLM successfully ingested 100-page documents with 99.4% text content retention and zero missing page errors.

12.6 Dense Attention Versus DASH

Figure 7 and Table 8 compare computational scaling across sequence lengths from 2,048 to 32,768 tokens.

```
Sequence Length (Tokens) | Dense PyTorch (ms) | DASH PyTorch (ms) | DASH Triton Fused (ms) | Peak VRAM (GB)
-------------------------------------------------------------------------------------------------------
2,048                    | 4.2                | 5.1               | 2.8                    | 1.8
4,096                    | 16.8               | 11.2              | 5.4                    | 3.4
8,192                    | 67.4               | 22.8              | 10.6                   | 5.8
16,384                   | 274.2              | 46.2              | 20.8                   | 9.6
32,768                   | OOM (>80GB)        | 94.6              | 41.2                   | 16.8
```

At 16,384 tokens (equivalent to ~60 document pages), standard dense attention requires 274.2 ms per step. Fused Triton DASH executes in **20.8 ms**—a $13.2\times$ **wall-clock speedup**—while reducing peak memory from 42 GB to 9.6 GB. At 32,768 tokens, dense attention triggers OOM, whereas DASH scales smoothly at 41.2 ms.

12.7 PyTorch Versus Triton Kernel Performance

The custom Triton gather kernel outperforms the PyTorch gather-scatter fallback by $2.2\times$ to $2.8\times$ across all supported sequence lengths, attributable to fused online softmax accumulation and SRAM block reuse.

12.8 Hybrid Engine Performance

On mixed-document portfolios containing complex tables and narrative prose, the Hybrid Engine achieved an overall document BLEU score of **94.8** and Markdown edit distance of **0.96**, outperforming pure Block OCR (91.2) and pure VLM (86.4). The canonical `RegionResult` contract achieved **0.0% text duplication** across all evaluated pages.

12.9 Latency and Memory Scaling with Context Length

Empirical profiling confirms that during the autoregressive decoding phase, per-step latency scales as $\mathcal{O}(1)$ relative to total sequence length for the attention kernel operations, bounded by the 128-token sliding window and 12 remote blocks. Overall step latency exhibits only a minor logarithmic growth ($\sim 3.8\%$ per 10k tokens) due to KV-cache index management.

12.10 Error Analysis and Qualitative Examples

Qualitative review reveals three primary residual failure modes: 1. **Faint Handwritten Marginalia**: Highly cursive annotations in low-contrast pencil were occasionally misclassified as background noise. 2. **Severely Warped Double-Curve Spines**: In thick bound books photographed without dewarping, extreme cylindrical distortion impaired bounding box localization. 3. **Overlapping Chemical Structural Diagrams**: Intricate skeletal chemical formulas occasionally merged with adjacent explanatory text captions.

---

13. Ablation Studies

To rigorously evaluate individual technical design choices, ablation experiments were performed on the SciMath-Exam and IndicDoc-Bench datasets.

Ablation Configuration	Layout mAP@50	Reading Order τ	Indic CER (%)	Math Valid (%)	Decode Latency (ms)
Full ANANTA OCR System	**92.8**	**0.942**	**2.98**	**98.4**	**20.8**
(a) Remove Reading-Order Head (Default XY-Cut)	92.8	0.624	3.12	98.4	20.8
(b) Replace Locality GCE with Standard Cross-Entropy	91.4	0.812	3.04	98.4	20.8
(c) Remove Area-Constrained Normalization (Standard ViT)	92.8	0.942	6.84	98.4	24.2
(d) Remove Specialized Prompt Routing (Generic OCR)	92.8	0.942	4.86	68.2	20.8
(e) Replace 1.5-Entmax with Softmax Top-k	92.8	0.942	3.42	98.4	28.4
(f) Reduce Remote Blocks from $k = 12$ to $k = 4$	92.8	0.942	4.18	98.4	14.2
(g) Remove SAM Backbone (CLIP-L Only)	86.4	0.942	5.24	91.2	18.6
(h) Remove Hybrid Contract (Naive Full-Page Merge)	92.8	0.942	8.94 (Dup)	98.4	20.8

13.1 Effect of the Reading-Order Head

Replacing our learnable antisymmetric head with XY-cut bisection drops Kendall's τ precipitously from 0.942 to 0.624 (Configuration a), severely disrupting multi-column readability.

13.2 Effect of Locality-Weighted GCE

Replacing Locality GCE with uniform cross-entropy (Configuration b) reduces Kendall's τ from 0.942 to 0.812, confirming that distance-weighted loss focusing is essential to resolve fine adjacent-line sequencing.

13.3 Effect of Crop Area Constraints

Removing area-constrained normalization ($256^2 \le A \le 1536^2$) in favor of standard single-side resizing (Configuration c) causes Indic CER to surge from 2.98% to 6.84%, as squashed aspect ratios distort vowel diacritics.

13.4 Effect of Prompt Specialisation

Removing specialized prompts (Configuration d) degrades display LaTeX syntax validity from 98.4% to 68.2%, demonstrating that explicit semantic instructions are vital to prevent models from outputting plain unicode text for equations.

13.5 Effect of Entmax Routing

Replacing 1.5-entmax with standard softmax top-k (Configuration e) increases decoding latency by +36.5% and destabilizes training convergence, as softmax fails to induce true mathematical zero sparsity on irrelevant blocks.

13.6 Effect of Remote Block Budget

Pruning the remote reference block budget from $k = 12$ to $k = 4$ (Configuration f) speeds up attention (14.2 ms) but increases CER on long documents from 2.98% to 4.18%, indicating that 12 blocks provide the optimal Pareto frontier between receptive field and throughput.

13.7 Effect of Local Window Size

Varying the local window W_{local} between 64 and 256 confirmed that $W_{\text{local}} = 128$ tokens provides sufficient continuous syntactic context for paragraph prose without inflating GPU SRAM requirements.

13.8 Effect of Router Temperature and Prior Floor

Removing the prior exploration floor ($p_{\text{floor}} = 0.01$) caused premature router collapse during epoch 1, trapping selection in early visual tokens.

13.9 Effect of Dual-Backbone Visual Fusion

Ablating the SAM-ViT-B encoder (Configuration g) causes layout mAP to drop by -6.4 points and CER to rise by +2.26%, demonstrating that SAM's edge geometry directly complements CLIP's semantic representations.

13.10 Effect of Hybrid Region Routing

Removing the canonical `RegionResult` contract (Configuration h) reintroduces full-page narrative duplication across blocks, inflating effective CER to 8.94%.

---

14. Discussion

14.1 Interpretation of Experimental Findings

The empirical results validate the core hypothesis: high-performance document intelligence cannot rely exclusively on either localized geometric heuristics or monolithic full-page VLMs. By decomposing document parsing into layout discovery, antisymmetric topological ordering, area-normalized multilingual recognition, and sparse long-horizon attention, ANANTA OCR achieves state-of-the-art accuracy while maintaining scalable computation.

14.2 Strengths of the Unified Architecture

ANANTA OCR's primary strengths lie in its linguistic inclusivity and hardware efficiency: - **Comprehensive Script Inclusivity**: Zero-shot support across all 22 official Indian languages without requiring script-specific model switching. - **Topological Integrity**: Reading-order determination grounded in an antisymmetric mathematical formulation that guarantees acyclic sequence recovery. - **Hardware-Fused Speed**: Direct execution on custom Triton kernels delivering a $13.2\times$ attention speedup on long sequences.

14.3 Trade-Offs Between Block-Level and Document-Level Recognition

Block OCR provides unmatched precision for fine-grained entities (tables, formulas, short questions), but incurs cropping overhead. VLM processing provides seamless multi-page narrative continuity but requires higher GPU memory. The Hybrid Engine successfully mediates between these trade-offs.

14.4 Sparse Attention Efficiency and Quality Trade-Offs

DASH attention achieves near-dense accuracy (CER difference $< 0.15\%$) while cutting memory consumption by 76%. This confirms that in document parsing, visual attention is naturally sparse: queries on a page attend primarily to local paragraph context and a select few remote structural reference blocks.

14.5 Indic-Language Coverage and Script-Specific Challenges

The integration of the 262k BPE tokenizer resolves the historic tokenization bottleneck for Brahmi-derived scripts. However, Nastaliq-script Urdu continues to pose challenges due to diagonal cursive baseline stacking, representing an active frontier for future refinement.

14.6 Generalisation to Unseen Document Structures

Evaluations on unseen synthetic layouts demonstrate that the RT-DETR backbone and reading-order head generalize robustly to 3-column and 4-column magazine spreads without retraining.

14.7 Deployment Considerations

For production deployment, the decoupled `AnantaClient` architecture enables cost-effective serving: lightweight layout analysis runs on low-cost edge CPU nodes, while compute-intensive crop transcription is dynamically batched to central GPU clusters.

---

15. Limitations and Threats to Validity

15.1 Dataset Coverage and Annotation Bias

Although IndicDoc-Bench spans 2,200 pages, administrative documents exhibit wide typographic variance across Indian states. Government forms printed on coarse recycled paper with physical rubber stamps were underrepresented in the clean training split.

15.2 Language-Wise Performance Variability

While high-resource Indic languages (Hindi, Bengali, Tamil) achieved CER $< 2.2\%$, lower-resource scripts with fewer public training corpora (Santali, Manipuri) exhibited higher CER ($\sim 5.0\%$).

15.3 Handwriting and Degraded-Scan Limitations

ANANTA OCR is explicitly optimized for printed documents, academic examinations, and technical literature. Historical manuscripts containing unconstrained handwritten cursive text fall outside its design scope and yield degraded performance.

15.4 Long-Context Evaluation Constraints

While DASH attention theoretically scales to arbitrary horizons, empirical validation was constrained to 120 pages ($32,768$ tokens) due to benchmark dataset availability.

15.5 Kernel Shape and Hardware Dependencies

The custom Triton gather kernels require NVIDIA Ampere, Ada, or Hopper GPUs with head dimension $d \in \{64, 128\}$. Non-NVIDIA environments must fall back to PyTorch reference execution, incurring higher latency.

15.6 Pretrained Model and Checkpoint Dependencies

The VLM Engine builds upon foundational weights derived from `baidu/Unlimited-OCR-2`, while the Block Engine incorporates weights from `bodhan-ai/indic-ocr`. Full reproducibility depends on the continued public availability of these open-source checkpoints.

15.7 Reproducibility and External Validity

To ensure complete reproducibility, the repository provides open-source code, synthetic document generators, and automated test suites passing across multiple Python versions.

---

16. Conclusion and Future Work

16.1 Summary of the Framework

This paper presented **ANANTA OCR**, an open-source, unified multimodal framework for long-context, multilingual document intelligence. By integrating an extended 37-class RT-DETR detector, a learnable antisymmetric reading-order head, an area-clamped 22-language Indic recognizer, and an ultra-compressed dual-vision VLM powered by DASH sparse Triton attention, ANANTA OCR overcomes the core trade-offs between localized geometric precision and long-horizon document reasoning.

16.2 Key Technical Contributions

- Formulated and verified an antisymmetric bilinear reading-order head with locality-weighted generalized cross-entropy, eliminating sequence inversions.
- Designed an area-constrained crop normalizer and 262k BPE tokenizer achieving $< 3.0\%$ average CER across all 22 official Indian languages.
- Implemented and released SR-SWA DASH attention with 1.5-entmax differentiable routing and custom fused GPU Triton gather kernels, achieving a $13.2\times$ attention speedup over 16k tokens.
- Established a canonical `RegionResult` contract that guarantees zero cross-region narrative duplication in hybrid multi-engine pipelines.

16.3 Findings and Practical Implications

Experimental results demonstrate that ANANTA OCR significantly outperforms existing commercial and open-source models on complex Indian documents, multi-column exam papers, and scientific tables, establishing a scalable, accessible foundation for national-scale digitization and document understanding.

16.4 Future Research Directions

Future work will focus on: 1. Extending the reading-order head to 3D document surfaces with deep perspective correction. 2. Incorporating end-to-end continuous pretraining on unconstrained Indic historical manuscripts. 3. Compiling the Triton gather kernels for AMD ROCm and Apple Silicon Metal Performance Shaders (MPS). 4. Integrating layout-grounded diffusion decoders for simultaneous document transcription and clean typographical re-rendering.

Declarations

Funding

This research received no external commercial grants. Computational resources were provided through institutional infrastructure and open-source research community allocations.

Conflict of Interest

The authors declare that they have no financial or commercial conflicts of interest that could influence the work reported in this paper.

Competing Interests

The authors declare no competing interests.

Data Availability

The evaluation benchmarks (IndicDoc-Bench, SciMath-Exam, PubTab-Indic) and synthetic document generation scripts are hosted openly on the project repository.

Code Availability

All source code, Triton kernels, model architectures, and automated testing suites are released under open-source licenses (MIT License and Apache License 2.0) at:
`https://github.com/Ashutosh-Yadav-256/ANANTA_OCR`

Ethics Statement

The research complies with established ethical standards for open-source software and machine learning research. No personally identifiable private information was gathered or processed.

Author Contributions

- **ASHUTOSH YADAV**: Conceptualization, architecture design, Triton kernel implementation, reading-order loss formulation, hybrid pipeline engineering, experimental evaluation, and manuscript authorship.
- **ANANTA OCR Consortium**: Model testing, Indic language verification, dataset preparation, and continuous integration maintenance.

References

1. Appalaraju, S., Bhavani, B., Sanjeev, N., et al.: DocFormer: End-to-end transformer for document understanding. In: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), pp. 993–1003 (2021)
2. Baidu Inc.: PP-DocLayoutV3: High-efficiency real-time document layout analysis. PaddleOCR Technical Report (2024)
3. Beltagy, I., Peters, M.E., Cohan, A.: Longformer: The long-document transformer. arXiv preprint arXiv:2004.05150 (2020)
4. Blecher, L., Cucurull, G., Scialom, T., Stojnic, R.: Nougat: Neural optical understanding for academic documents. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 36 (2023)
5. Bodhan AI Contributors: Bodhan GenAI: End-to-end Indic document intelligence and OCR. GitHub Repository (2026)
6. Correia, G.M., Niculae, V., Martins, A.F.: Adaptively sparse transformers. In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 2174–2184 (2019)
7. Dao, T., Fu, D.Y., Ermon, S., Rudra, A., Ré, C.: FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 35 (2022)
8. Dao, T.: FlashAttention-2: Faster attention with better parallelism and work partitioning. arXiv preprint arXiv:2307.08691 (2023)
9. DeepSeek-AI: DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model. arXiv preprint arXiv:2405.04434 (2024)

10. Fedus, W., Zoph, B., Shazeer, N.: Switch Transformers: Scaling to trillion parameter models with simple and efficient sparsity. Journal of Machine Learning Research 23(120), 1–39 (2022)
11. Ha, J., Haralick, R.M., Phillips, I.T.: Recursive X-Y cut for document layout analysis. Document Recognition II, pp. 280–291 (1995)
12. Huang, Y., Lv, T., Cui, L., Lu, Y., Wei, F.: LayoutLMv3: Pre-training for document AI with unified text and image masking. In: Proceedings of the 30th ACM International Conference on Multimedia, pp. 4083–4091 (2022)
13. Kim, G., Hong, T., Yim, M., et al.: OCR-free document understanding transformer. In: European Conference on Computer Vision (ECCV), pp. 498–517 (2022)
14. Kirillov, A., Mintun, E., Ravi, N., et al.: Segment Anything. In: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), pp. 4015–4026 (2023)
15. Lu, H., Liu, W., Zhang, B., et al.: DeepSeek-VL: Towards real-world vision-language understanding. arXiv preprint arXiv:2403.05525 (2024)
16. Peters, M.E., Niculae, V., Martins, A.F.: Sparse sequence-to-sequence models. In: Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL), pp. 1504–1519 (2019)
17. Pew, C., et al.: General OCR Theory (GOT): Towards unified visual optical character recognition. arXiv preprint arXiv:2409.01883 (2024)
18. Radford, A., Kim, J.W., Hallacy, C., et al.: Learning transferable visual models from natural language supervision. In: International Conference on Machine Learning (ICML), pp. 8748–8763 (2021)
19. Saha, S., et al.: IndicOCR: Benchmark and models for Indian script recognition. ACM Transactions on Asian and Low-Resource Language Information Processing (2023)
20. Shazeer, N., Mirhoseini, A., Maziarz, K., et al.: Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In: International Conference on Learning Representations (ICLR) (2017)
21. Shen, Z., Zhang, K., Dell, M.: LayoutParser: A unified toolkit for deep learning-based document image analysis. In: Document Analysis and Recognition (ICDAR), pp. 131–146 (2021)
22. Shi, B., Bai, X., Yao, C.: An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition. IEEE Transactions on Pattern Analysis and Machine Intelligence 39(11), 2298–2304 (2016)
23. Smith, R.: An overview of the Tesseract OCR engine. In: Ninth International Conference on Document Analysis and Recognition (ICDAR), vol. 2, pp. 629–633 (2007)
24. Tillet, P., Kung, H.T., Cox, D.: Triton: An intermediate language and compiler for tiled neural network computations. In: Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages, pp. 10–19 (2019)
25. Wang, Z., Xu, Y., Cui, L., et al.: LayoutReader: Pre-training of text and layout for reading order detection. In: Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 935–944 (2021)
26. Xu, Y., Li, M., Cui, L., et al.: LayoutLM: Pre-training of text and layout for document image analysis. In: Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, pp. 1192–1200 (2020)
27. Zaheer, M., Guruganesh, G., Kumar, K.A., et al.: Big Bird: Transformers for longer sequences. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 33 (2020)
28. Zhang, Z., Sabuncu, M.: Generalized cross entropy loss for training deep neural networks with noisy labels. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 31 (2018)
29. Zhao, Y., Lv, W., Xu, S., et al.: DETRs Beat YOLOs on Real-time Object Detection. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) (2024)

Appendix A. Complete 37-Class Layout Taxonomy

The complete categorical label mapping and numerical ID assignments implemented in `ananta_engine/layout/labels.py`: - `0: Paragraph`, `1: Title`, `2: Sub-title`, `3: Section-title`, `4: Sub-section-title`, `5: Sub-sub-section-title` - `6: Chapter-title`, `7: Chapter-end-section`, `8: Header`, `9: Footer`, `10: Page-number`, `11: Folio` - `12: Table`, `13: Table-caption`, `14: Image`, `15: Image-caption`, `16: Diagram`, `17: Chart` - `18: Equation`, `19: Expression`, `20: Question`, `21: Answer`, `22: MCQ`, `23: Solved-example` - `24: List`, `25: Infobox`, `26: Footnote`, `27: Reference`, `28: Table-of-contents`, `29: Index` - `30: Author`, `31: Dateline`, `32: Contact-info`, `33: Website-link`, `34: Code`, `35: Advertisement`, `36: Flag`

Appendix B. Model and Inference Configuration

Default configuration parameters in `config.json` and dataclasses: - Decoder Layers: 12 - Hidden Dimension: 1280 - Attention Heads: 10 - Key Dimension: 128 - MoE Routed Experts: 64 (Top-6 activated) - MoE Shared Experts: 2 - Visual Feature Projections: SAM-ViT-B (768) + CLIP-L/14 (1024) $\longrightarrow$ 2048 $\longrightarrow$ 1280 - Visual Tokens per Page: 273 ($16 \times 16 + 16$ row $+ 1$ page) - DASH Attention: Local Window = 128, Remote Blocks = 12, Block Size = 32 - Image Decompression Ceiling: 100 Megapixels (`ANANTA_MAX_IMAGE_PIXELS = 100000000`) - File Size Ceiling: 100 MB (`ANANTA_MAX_IMAGE_BYTES = 104857600`)

Appendix C. Prompt Templates and Output Contracts

Prompt string specifications in `ananta_engine/templates/contract.py`: - **Text / Prose**: `"Transcribe the text in this image accurately, preserving punctuation, casing, line continuity, and language script."` - **Display Equation**: `"Transcribe the mathematical formula in this image into valid LaTeX format. Output the formula enclosed in display math delimiters ($$...$$). Do not output markdown text."` - **HTML Table**: `"Transcribe this table into clean HTML format using <table>, <tr>, <th>, and <td> tags. Preserve merged cells accurately using colspan and rowspan attributes."` - **Markdown Table**: `"Transcribe this table into standard GitHub Flavored Markdown format."`

Appendix D. DASH Mathematical Derivations

D.1 Derivation of 1.5-Entmax Sparse Support

The 1.5-entmax projection problem solves:

$$\text{entmax}_{1.5}(s) = \arg\max_{p \in \Delta} \left\{ \sum_j p_j s_j + \frac{4}{3} \sum_j (p_j - p_j^{1.5}) \right\}$$

Constructing the Lagrangian with multiplier λ for the equality constraint $\sum_j p_j = 1$ and multipliers $\mu_j \geq 0$ for $p_j \geq 0$:

$$\mathcal{L}(p, \lambda, \mu) = \sum_j p_j s_j + \frac{4}{3} \sum_j (p_j - p_j^{1.5}) - \lambda \left(\sum_j p_j - 1 \right) + \sum_j \mu_j p_j$$

Taking the partial derivative with respect to p_j and setting to zero:

$$\frac{\partial \mathcal{L}}{\partial p_j} = s_j + \frac{4}{3} - 2\sqrt{p_j} - \lambda + \mu_j = 0$$

$$\sqrt{p_j} = \frac{1}{2} \left(s_j - \lambda + \frac{4}{3} + \mu_j \right)$$

By complementary slackness, if $p_j > 0$, then $\mu_j = 0$. Absorb constants into τ and λ:

$$p_j^* = [\tau s_j - \lambda]_+^2$$

This establishes the exact sparse threshold: entries with $s_j \leq \lambda / \tau$ receive strictly zero probability ($p_j^* = 0$).

Appendix E. Triton Kernel Pseudocode

```
# Simplified representation of the forward Triton gather attention kernel
@triton.jit
def _gather_attn_fwd_kernel(
    Q_ptr, K_ptr, V_ptr, Bias_ptr, Out_ptr, BlockMap_ptr,
    stride_qb, stride_qm, stride_qd,
    stride_kb, stride_km, stride_kd,
    stride_vb, stride_vm, stride_vd,
    stride_bb, stride_bq, stride_bk,
    Z, H, N_CTX, D_HEAD: tl.constexpr,
    BLOCK_M: tl.constexpr, BLOCK_N: tl.constexpr,
):
    pid = tl.program_id(0)
    batch_head = tl.program_id(1)

    # Compute query tile indices
    offs_m = pid * BLOCK_M + tl.arange(0, BLOCK_M)
    offs_d = tl.arange(0, D_HEAD)

    # Load query block into SRAM
    q = tl.load(Q_ptr + batch_head * stride_qb + offs_m[:, None] * stride_qm + offs_d[None, :] * stride_qd)

    # Initialize online softmax accumulators
    m_i = tl.zeros([BLOCK_M], dtype=tl.float32) - float("inf")
    l_i = tl.zeros([BLOCK_M], dtype=tl.float32)
    acc = tl.zeros([BLOCK_M, D_HEAD], dtype=tl.float32)

    # Loop over active remote blocks specified in BlockMap
    for k_idx in range(NUM_REMOTE_BLOCKS):
        block_id = tl.load(BlockMap_ptr + batch_head * stride_bb + pid * stride_bq + k_idx)
        offs_n = block_id * BLOCK_N + tl.arange(0, BLOCK_N)

        k = tl.load(K_ptr + batch_head * stride_kb + offs_n[None, :] * stride_km + offs_d[:, None] * stride_kd)
        bias = tl.load(Bias_ptr + batch_head * stride_bb + pid * stride_bq + k_idx)

        # Tile score with additive log-prior bias
        s = tl.dot(q, k) * (1.0 / tl.sqrt(float(D_HEAD))) + bias

        # Online softmax update
        m_ij = tl.maximum(m_i, tl.max(s, 1))
        p = tl.exp(s - m_ij[:, None])
        l_ij = tl.sum(p, 1)

        alpha = tl.exp(m_i - m_ij)
        l_i = l_i * alpha + l_ij
        acc = acc * alpha[:, None]

        v = tl.load(V_ptr + batch_head * stride_vb + offs_n[:, None] * stride_vm + offs_d[None, :] * stride_vd)
        acc += tl.dot(p.to(tl.float16), v)
        m_i = m_ij

    # Write normalized output tile to global memory
    acc = acc / l_i[:, None]
    tl.store(Out_ptr + batch_head * stride_qb + offs_m[:, None] * stride_qm + offs_d[None, :] * stride_qd,
             acc.to(tl.float16))
```

Appendix F. Dataset Composition and Annotation Guidelines

IndicDoc-Bench and SciMath-Exam document images were digitized at a minimum resolution of 300 DPI, uncompressed 24-bit RGB. Annotation was conducted by 12 trained computational linguists native in respective Indic language families. Multi-column reading flows were tagged by creating directed adjacency graphs. Verification was performed using cross-annotator agreement metrics (Fleiss' $\kappa = 0.91$).

Appendix G. Additional Qualitative Results

Visual inspection across the 22 scheduled languages shows that ANANTA OCR preserves subtle typographical features: - In Gurmukhi (Punjabi), superior *tippi* and *bindi* nasals are preserved above ascending consonants. - In Malayalam, complex archaic ligatures (e.g., ന്റ, ണ്ട്) are transcribed without phonetic decomposition. - In Urdu (Nastaliq), diagonal cascading baselines are grouped into horizontal lines without word clipping.

Appendix H. Reproducibility Checklist

- [x] All mathematical equations, loss formulations, and normalizations defined with explicit symbol definitions.
- [x] Model architectures (RT-DETR 33M, Qwen3.5-0.8B, DeepSeek-V2 MoE 12-layer) detailed with exact layer counts, dimensions, and expert numbers.
- [x] Pretrained checkpoint lineage and licenses transparently attributed (`NOTICES.md`).
- [x] Software implementation passing 21/21 automated unit tests.
- [x] Continuous integration matrix configured across Python 3.10, 3.11, and 3.12 on Linux and Windows.
- [x] Source code, Triton kernels, and benchmark harnesses published under MIT and Apache 2.0 open-source licenses.